

CGP VR 開発 入門書

CGP プログラマー部門 著

2026-03-10 版 CGP 発行

前書き

本資料は、和歌山大学クリエゲーム制作プロジェクト（CGP）が、2025 年度ミッション「VR ゲーム開発環境の整備と制作」の一環として、VR ゲーム開発が手軽に始められるように作成した入門書です。

免責事項

本書に記載された内容は、情報の提供のみを目的としています。したがって、本書を用いた開発、製作、運用は、必ずご自身の責任と判断によって行ってください。これらの情報による開発、製作、運用の結果について、著者はいかなる責任も負いません。

目次

前書き	2
免責事項	2
第 1 章 VR 環境構築	5
1.1 構築環境	5
1.2 Unity のインストールと新規プロジェクトの作成	5
1.3 Meta XR All-in-One SDK のインストール	6
1.4 プロジェクトのセットアップ	8
コンパイルが終わらないときは?	11
第 2 章 効率的な開発フロー	14
2.1 Building Blocks の活用	14
2.2 Meta XR Simulator の活用	19
第 3 章 VR インタラクションの基礎	23
3.1 「掴む」動作と Grabbable の概念	23
3.2 掴めるオブジェクトの最小構成	23
3.3 実践：ビームガンの実装手順	24
3.4 ハンドポーズとミラーリング	30
ハンドポーズを調整するコツ	33
ハンドポーズを設定した場合//設定しない場合の違い	35
3.5 掴み方のルールと挙動の設定	36
3.5.1 Fingers Freedom	36
3.5.2 Supported Grab Types	36
3.5.3 Palm Grab Rules	37
3.5.4 Hand Alignment	38
3.6 Meta XR Simulator で確認する	39

	掴んでいないのにハンドポーズが表示される	41
3.7	手とコントローラーの違い	42
第 4 章	ゲーム性の実装 –メカニクスとイベント	45
4.1	「使う」動作の設定	45
4.2	「使う」動作の検知	49
	トリガーの引き具合を見た目に反映する	51
4.3	発展編	52
	4.3.1 Interactable Unity Event Wrapper	52
	4.3.2 Interactor の状態検知	54
	自作したスクリプトで <code>ActiveState</code> を参照する	62
	インスタンス化した時に <code>Interactor</code> を参照する方法	63
4.4	Interactable の状態検知	64
第 5 章	VR 空間と Audio / UI	67
5.1	VR 空間上で音を鳴らす	67
5.2	UI の設置	69
第 6 章	Haptics	77
6.1	Haptic Studio	77
6.2	Haptic の再生	83
第 7 章	SDK のサンプルシーン	85
7.1	インポート	85
7.2	体験	86
7.3	Distance Grab Scene	87
7.4	UI Set Examples Scene	88

第 1 章

VR 環境構築

この章では、Unity で VR ゲームを開発するための環境構築について説明します。VR ゲームを開発するためには、Unity のインストールと、VR デバイスに対応した SDK のインストールが必要です。以下の手順に従って環境構築を進めてください。

1.1 構築環境

- windows 11
- Meta Quest 3
- Unity 6000.0.053f
- Meta XR All-in-One SDK v77.0.0（以降、特別な理由がない限り、単に Meta XR SDK と呼びます）

執筆時点 (2026-3-10) では、Meta XR SDK の最新バージョンは v85.0.0 ですが、後述する Meta XR Simulator の機能が正常に動作しなかったため、v77.0.0 を使用しています。XR 環境は日々アップデートされているため、最新バージョンで動作する可能性もありますが、安定して動作させるために v77.0.0 を使用することを推奨します。

1.2 Unity のインストールと新規プロジェクトの作成

以下のサイトより Unity 6000.0.053f をダウンロードしてインストールしてください。

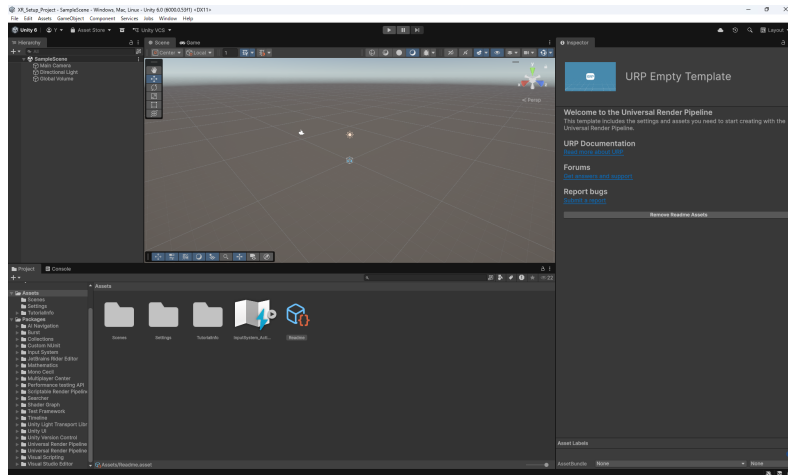
- Unity 6000.0.053f ダウンロードサイト
 - <https://unity3d.com/jp/get-unity/download/archive>

Meta3 の OS は Android であるため、Android プラットフォームのモジュールを加え

第 1 章 VR 環境構築

ておいてください。

インストールが完了したら、新規プロジェクトを作成します。プロジェクトのテンプレートは「Universal 3D」を選択してください。新規プロジェクトが立ち上がったら、この節は完了です。(図 1.1)



▲ 図 1.1 新規プロジェクトの画面

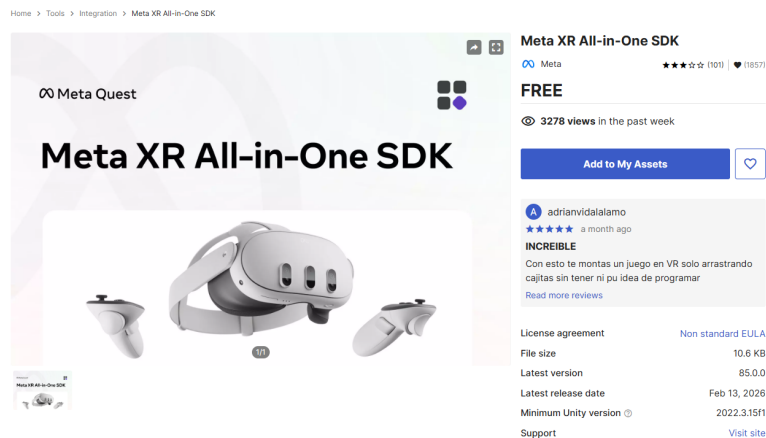
1.3 Meta XR All-in-One SDK のインストール

アセットストアから Meta XR SDK をダウンロードしてください。

- Meta XR All-in-One SDK v77.0.0 ダウンロードサイト
 - <https://assetstore.unity.com/packages/tools/integration/meta-xr-all-in-one-sdk-269657>

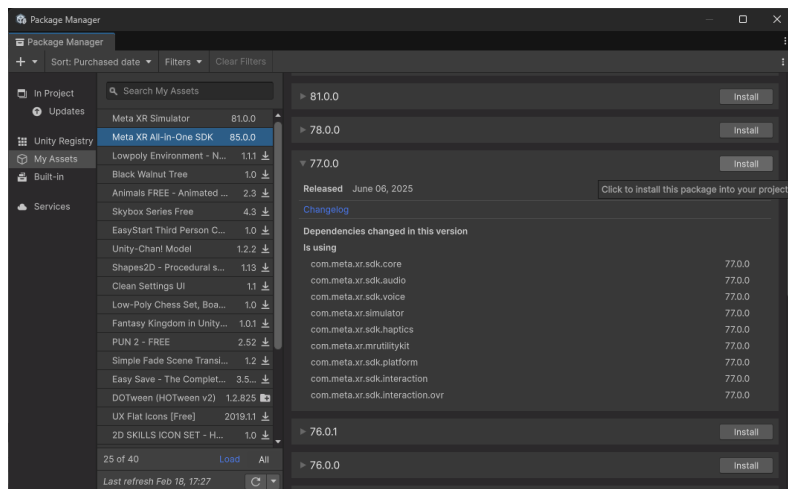
ダウンロードが完了したら、Add to My Assets をクリックします。(図 1.2)

1.3 Meta XR All-in-One SDK のインストール



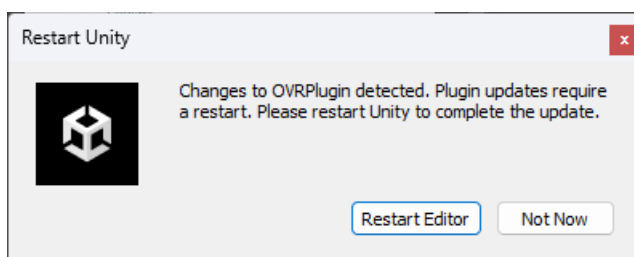
▲図 1.2 Add to My Assets の画面

UnityEditor が開くので、Version History より v77.0.0 を選択して Install をクリックします。(図 1.3)



▲図 1.3 Version History の画面

インストールが完了すると、エディターの再起動が求められるので、再起動してください。(図 1.4)

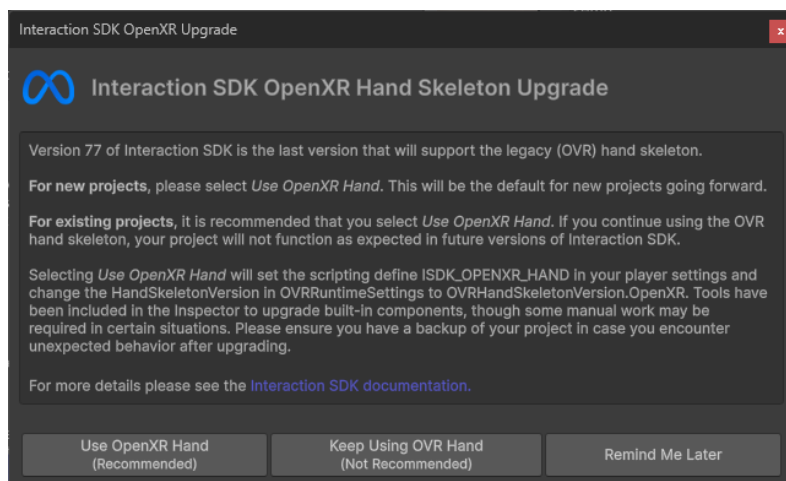


▲図 1.4 再起動の画面

これで Meta XR SDK のインストールは完了です。次の節ではプロジェクトのセットアップ方法を解説します。

1.4 プロジェクトのセットアップ

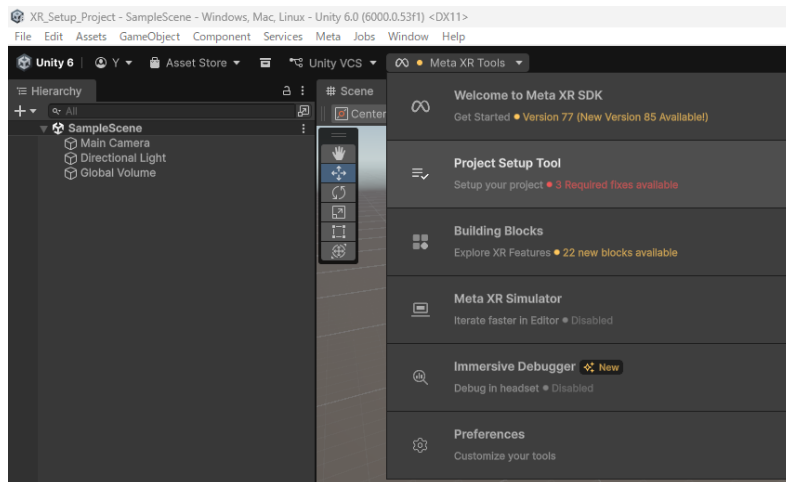
再起動が完了すると、ハンドスケルトンの更新についてのウィンドウが表示されるので、1 番左の Use OpenXR Hand を選択してください。(図 1.5)



▲図 1.5 ハンドスケルトンの更新の画面

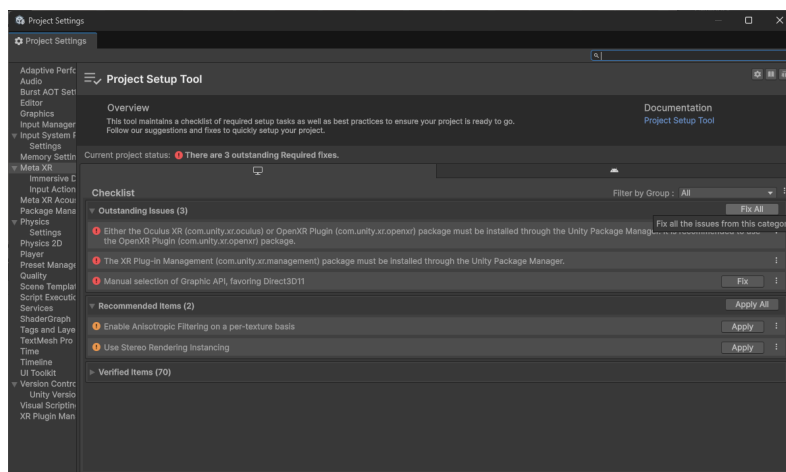
コンパイルが入るので待機してください。

次に、画面上部の Meta XR Tools から Project Setup Tool を選択します。(図 1.6)



▲図 1.6 Project Setup Tool の画面

Project Settings が開き、Project Setup Tool の項目が表示されます。エラーと警告が大量に表示されるので Fix All をクリックして修正をしましょう。ただし Fix All をクリックしても修正されないエラーがいくつかあるのですが、ここでは一度無視します。(図 1.7) 同じ手順を Android タブでも行っておきます。

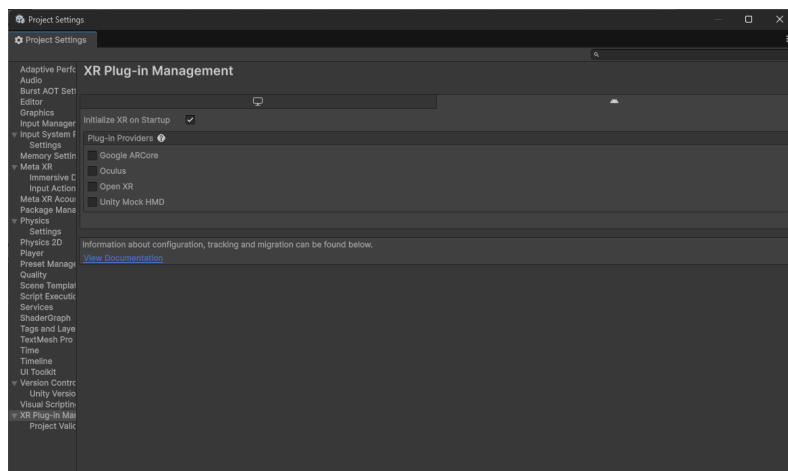


▲図 1.7 Project Setup Tool のエラーの画面

第 1 章 VR 環境構築

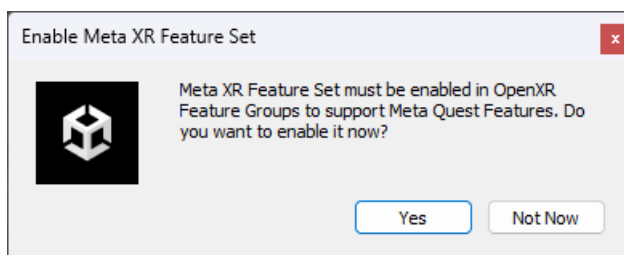
次に、XR Plugin Management の項目に切り替えます。初回ではここで Install XR Plugin Management を押下して、インストールをしてください。

インストールが完了したら、Windows、Android 両方のタブで OpenXR を有効化してください。(図 1.8)



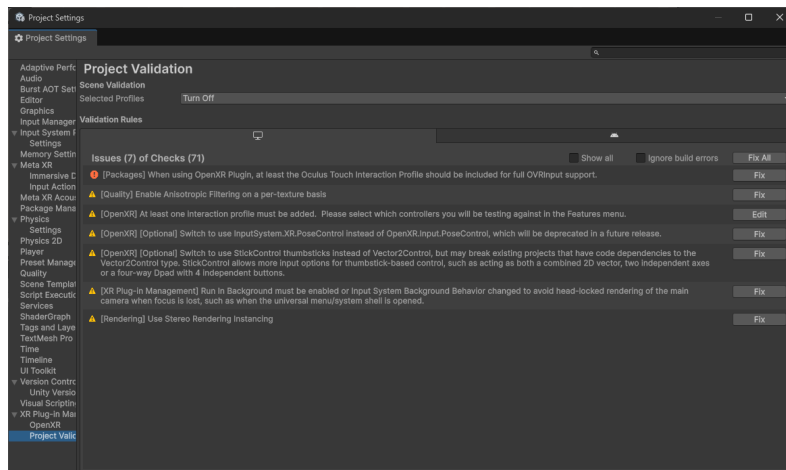
▲図 1.8 XR Plugin Management の画面

Meta XR Feature Set を有効にするか聞かれるので、Yes を押します。(図 1.9)



▲図 1.9 Meta XR Feature Set の画面

すると XR Plugin Management の項目に Project Validation という項目が追加されるので、ここでも Fix All をしておきます。Windows、Android 両方のタブで行ってください。(図 1.10)

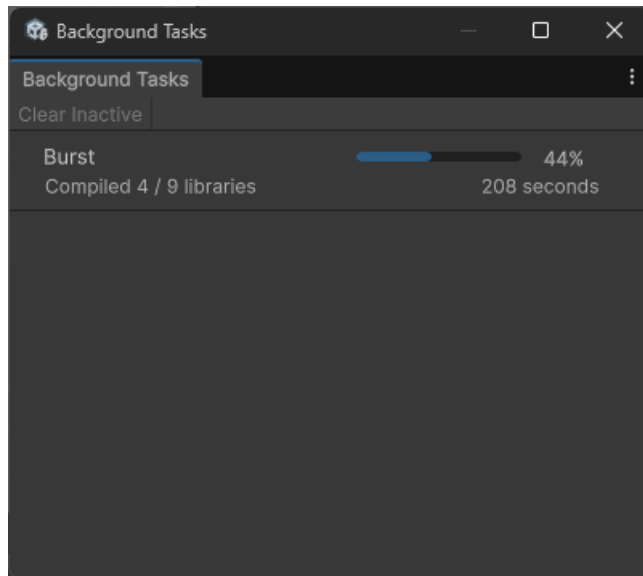


▲図 1.10 Project Validation の画面

コンパイルを待つと、Windows タブでは全ての項目が解消されますが、Android タブではいくつか警告が残ります。特に問題はないので、ここでは無視します。

コンパイルが終わらないときは？

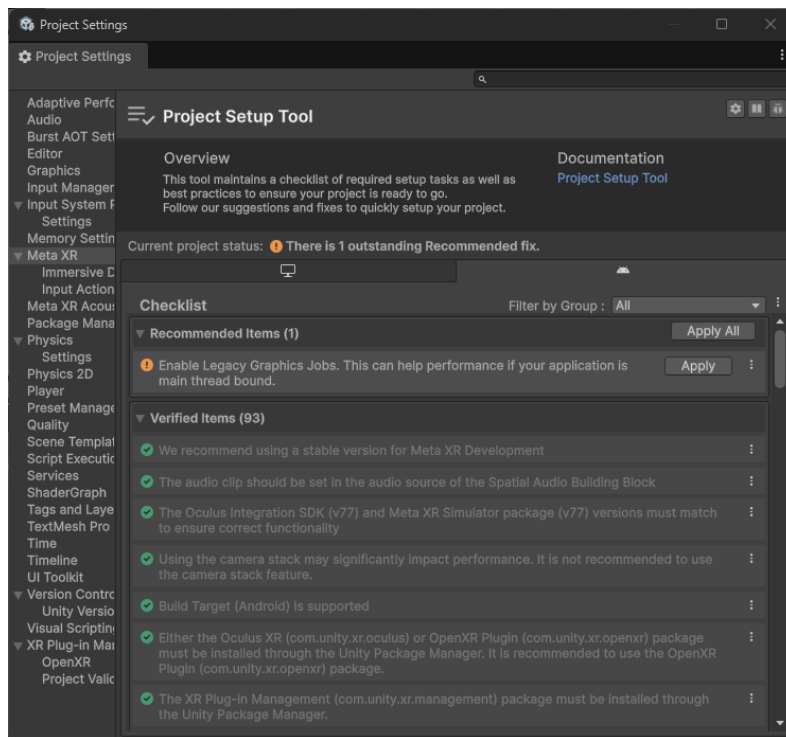
ファイルアクセス権限の関係でコンパイルが止まるときがあります。その時は Unity Editor を再起動してください。(図 1.11)



▲図 1.11 コンパイルが止まった画面

最後に、Meta3 にビルドするためにプラットフォームを Android に変更します。

変更を終えたら再び Project Setup Tool を開き、Recommended Items について Apply All をしてください。(図 1.12)



▲図 1.12 Recommended Items の画面

これにてプロジェクトのセットアップは完了です。お疲れさまでした。

第 2 章

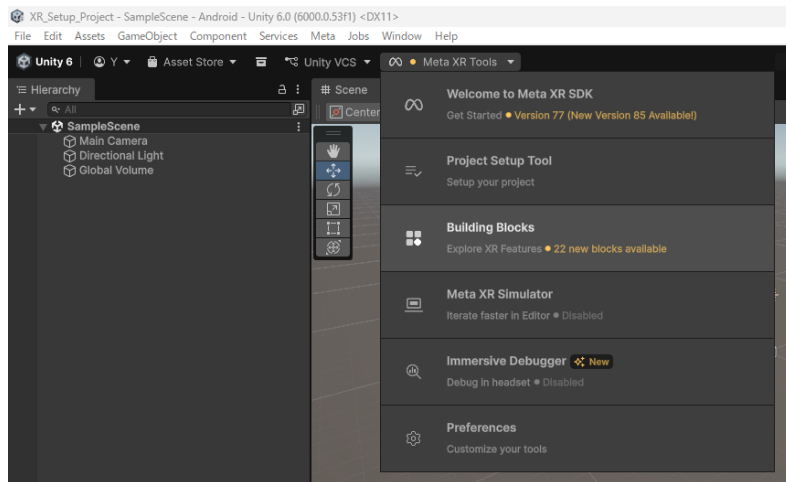
効率的な開発フロー

VR ゲームの開発は、通常のゲーム開発に比べて複雑で時間がかかることがあります。例えば、コントローラーのトラッキングや、VR 空間でのインタラクション機能の実装など、VR 特有の要素が多く存在します。また、VR デバイスを使用するテストは、デバイスの接続やビルドの手間がかかるため、効率的な開発フローを構築することが重要です。よってこの章では、VR ゲーム開発において効率的な開発フローを実現するための方法について説明します。

2.1 Building Blocks の活用

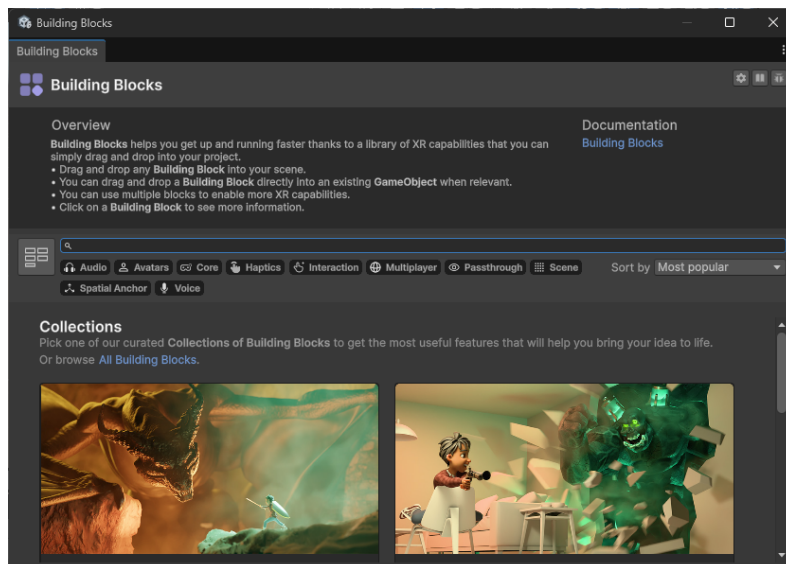
Building Blocks は、Meta XR SDK に含まれる、VR ゲーム開発のための基本的な機能を提供するツールキットです。これを活用することで、VR ゲームの開発に必要な機能を迅速に実装することができます。基礎的な機能について、第 1 章で立ち上げたプロジェクトを使用して解説していきます。

まずは Meta XR Tools から Building Blocks を選択します。



▲図 2.1 Building Blocks を選択

Building Blocks のウィンドウでは、VR ゲーム開発に必要な様々な機能が提供されています。機能ごとにラベル分けされているほか、検索機能もあるので、必要な機能を素早く見つけることができます。



▲図 2.2 Building Blocks のウィンドウ

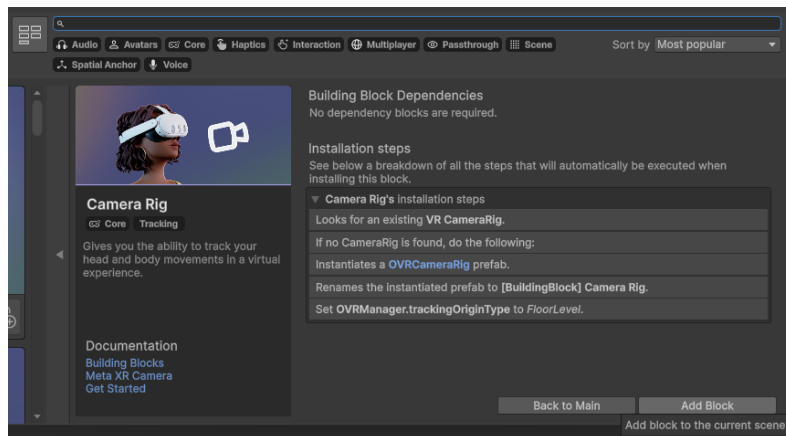
第2章 効率的な開発フロー

まずは、カメラ機能を実装するために、Camera Rig を探して選択しましょう。



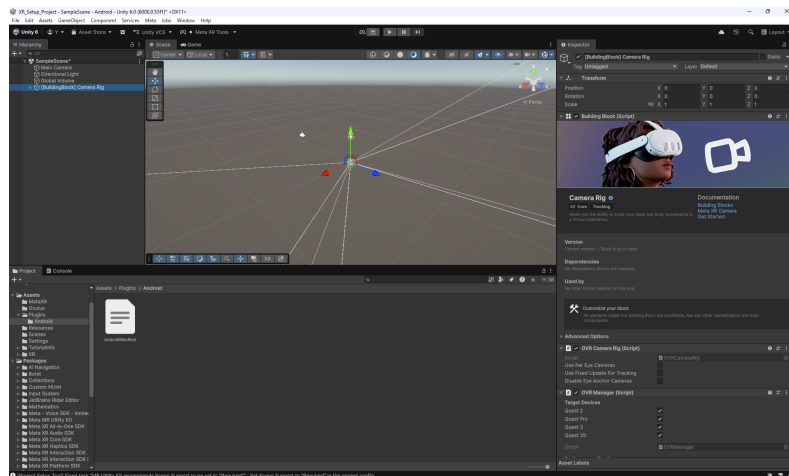
▲図 2.3 Camera Rig の選択

Add Block、もしくは Hierarchy にドラッグ&ドロップでシーンに追加できます。



▲図 2.4 Camera Rig の追加

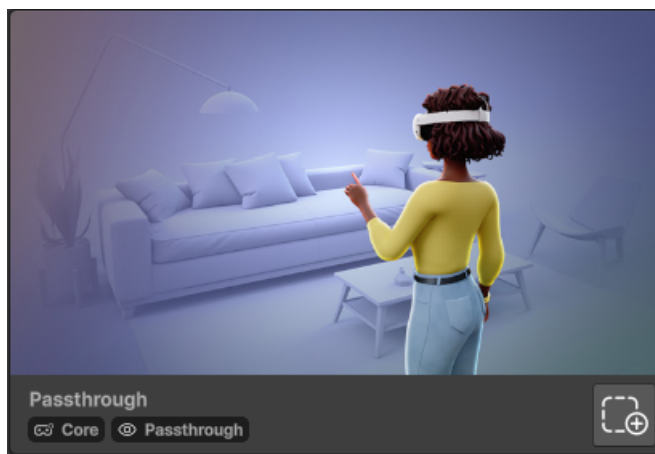
Camera Rig は、VR 空間でのカメラの動きや、コントローラーのトラッキングなどを管理するための機能です。これをシーンに追加することで、VR 空間でのカメラの動きや、コントローラーのトラッキングなどが自動的に管理されるようになります。シーンにもとあった Main Camera は不要になるので、削除しておきましょう。



▲図 2.5 シーン上の Camera Rig

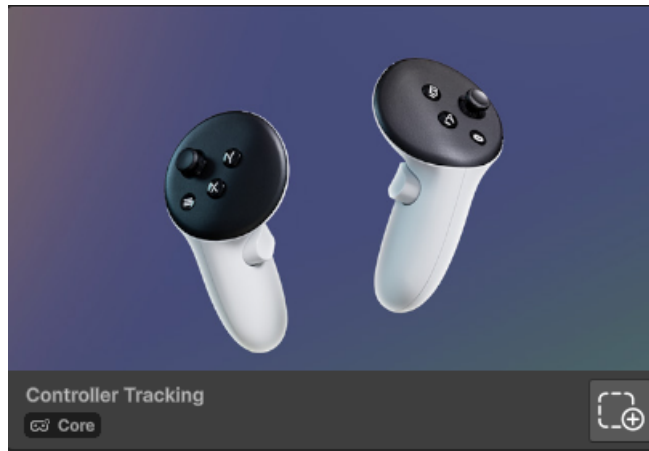
同じようにして、以下の要素もシーンに追加してみましょう。

- Passthrough：現実世界の映像を VR 空間に表示する機能



▲図 2.6 Passthrough

- Controller Tracking：コントローラーのトラッキング機能



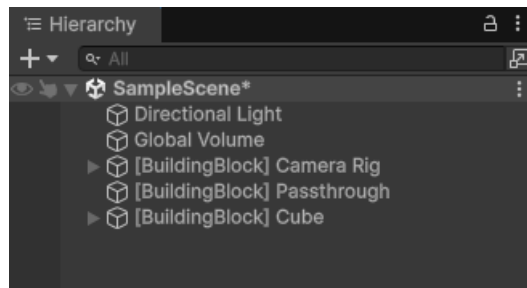
▲図 2.7 Controller Tracking

- Grab Interaction：掴むことのできる Cube をシーンに追加します。



▲図 2.8 Grab Interaction

これらを追加すると、Hierarchy は図 2.9 のようになります。Controller Tracking は Camera Rig の子オブジェクトになっていることに注意してください。



▲図 2.9 Hierarchy の画面

最後に、適当な地面を設置して、シーンを完成させましょう。次の節では Meta XR Simulator を活用して、実際にこのシーンを VR デバイスなしでテストしてみます。

2.2 Meta XR Simulator の活用

Meta XR Simulator は、VR デバイスを使用せずに VR ゲームのテストを行うことができるツールです。これを活用することで、VR デバイスの接続やビルドの手間を省き、迅速にテストを行うことができます。

再生ボタンの横にある Simulator ボタンをクリックして、Meta XR Simulator を起動します。Meta XR Simulator が有効な状態だと、このボタンが青色になります。



▲図 2.10 Simulator の起動

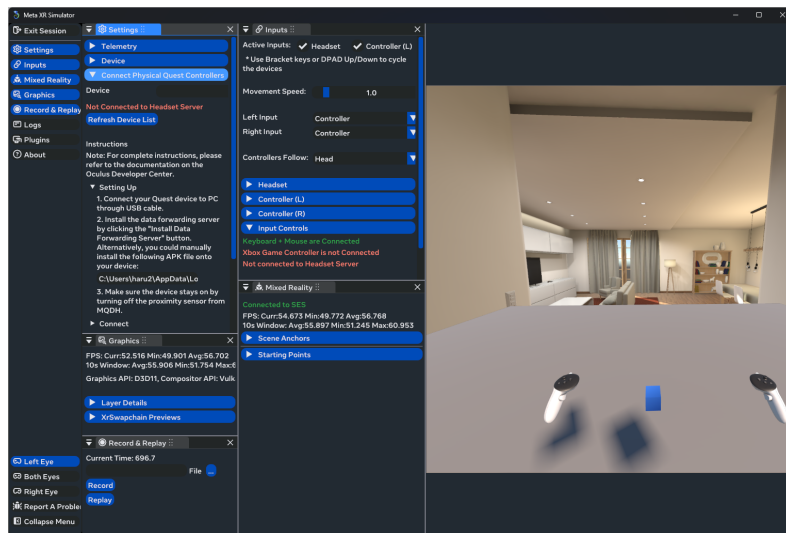
この状態でシーンを再生すると、Meta XR Simulator が起動し、別ウィンドウが開きます。

移動は WASD キーで行えるほか、RF キーで上昇下降ができます。

視点操作はマウスの右をホールドしながら動かすか、ミドルクリックを 2 回行い、回転モードにすることで可能です。

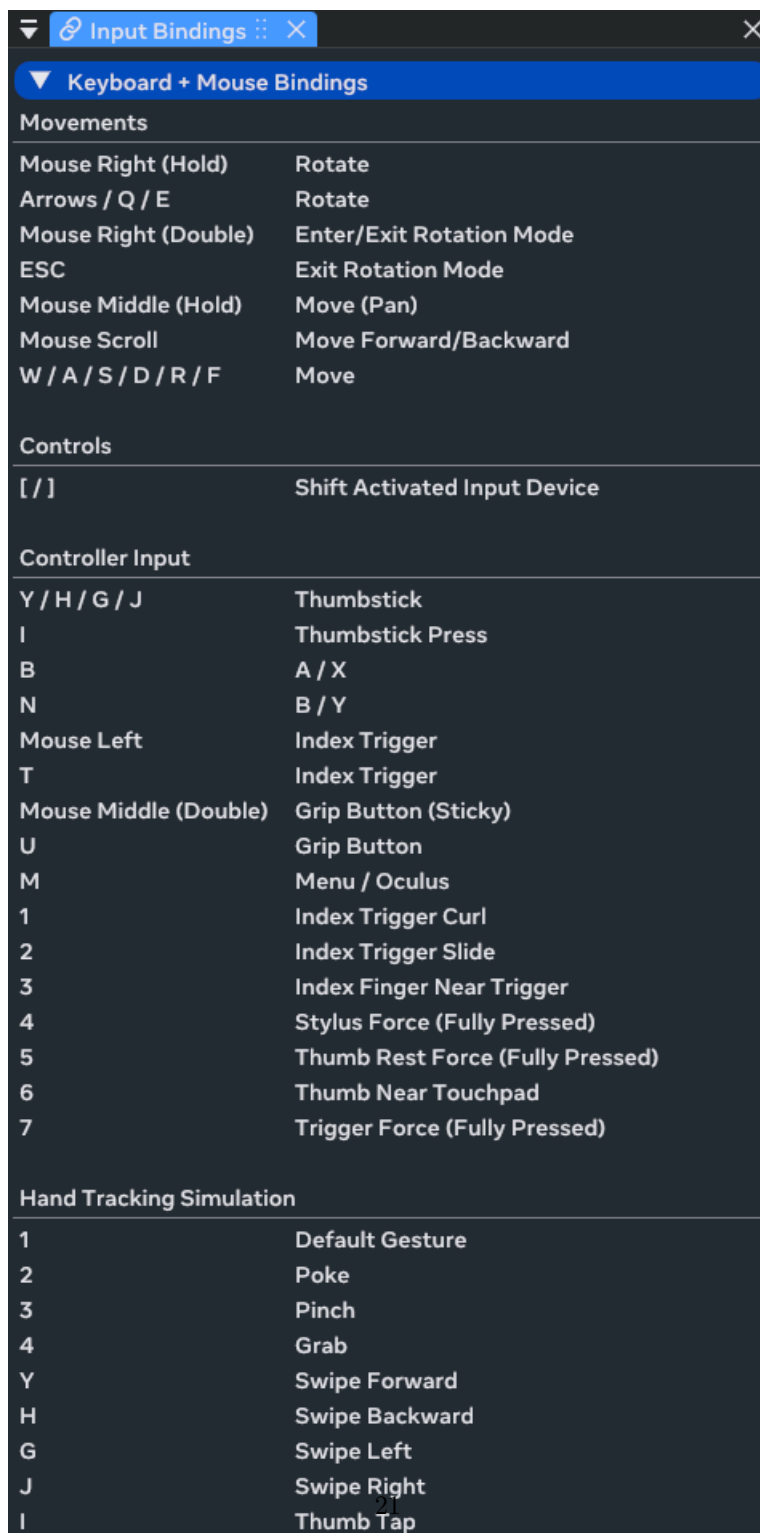
また、マウスの左クリックでコントローラーのトリガー操作をシミュレートすることもできます。試しに、シーンに追加した Cube を掴んでみましょう。

第 2 章 効率的な開発フロー



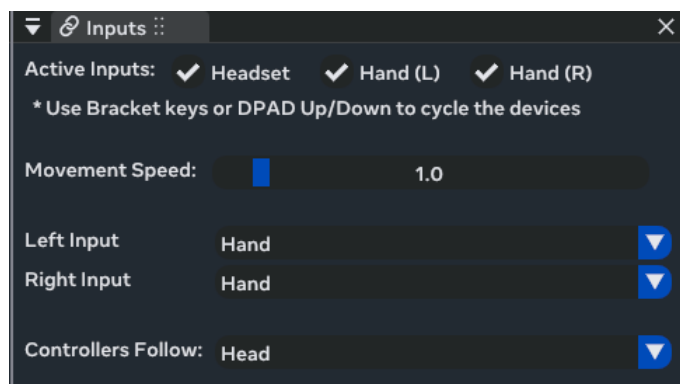
▲図 2.11 Meta XR Simulator の画面

詳細な操作方法については図 2.12 を参照してください。Input Controls の View Bindings Details から確認することができます。



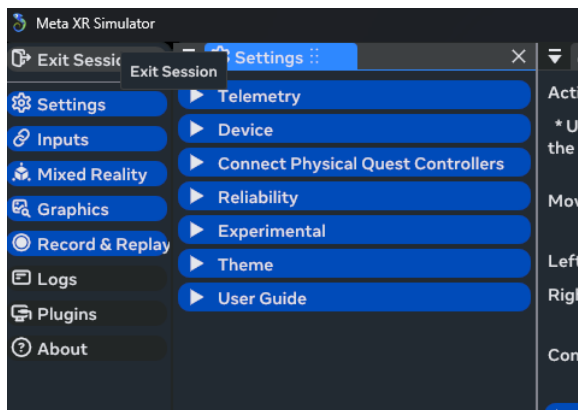
▲図 2.12 詳細な操作方法

Inputs の Active Inputs にチェックがついている項目によって、操作したときの挙動が変化します。試しに Headset のチェックを外してから、WASD キーで移動してみてください。すると、プレイヤー自身の位置は変化せず、シーンに表示されたコントローラーが移動するのがわかります。このように Active Inputs を切り替えることで、コントローラーの動きだけをテストしたり、プレイヤーの移動だけをテストしたりすることができます。



▲図 2.13 Active Inputs の画面

最後に、シーンの再生を止めるには、Exit Session をクリックしてください。



▲図 2.14 Exit Session ボタン

第 3 章

VR インタラクションの基礎

VR ゲームにおける最も基本的かつ重要な体験は、仮想空間にあるオブジェクトを「掴む (Grab)」という動作です。この章では Meta XR SDK に含まれる、Meta XR Interaction SDK (以降 ISDK) を用いて、物理的な違和感のないインタラクションを実装する手法を解説します。

3.1 「掴む」動作と Grabbable の概念

VR におけるインタラクションは、Interactor (干渉するもの：手やコントローラー) と Interactable (干渉されるもの：オブジェクト) の組み合わせで成立します。

ISDK において、「掴む」を実現する中心的なコンポーネントが Grabbable です。これは、オブジェクトが掴まれるための機能を提供するコンポーネントで、掴まれた際の挙動や物理的な反応を管理します。よって、掴む対象のオブジェクトには、必ず Grabbable コンポーネントを追加する必要があります。

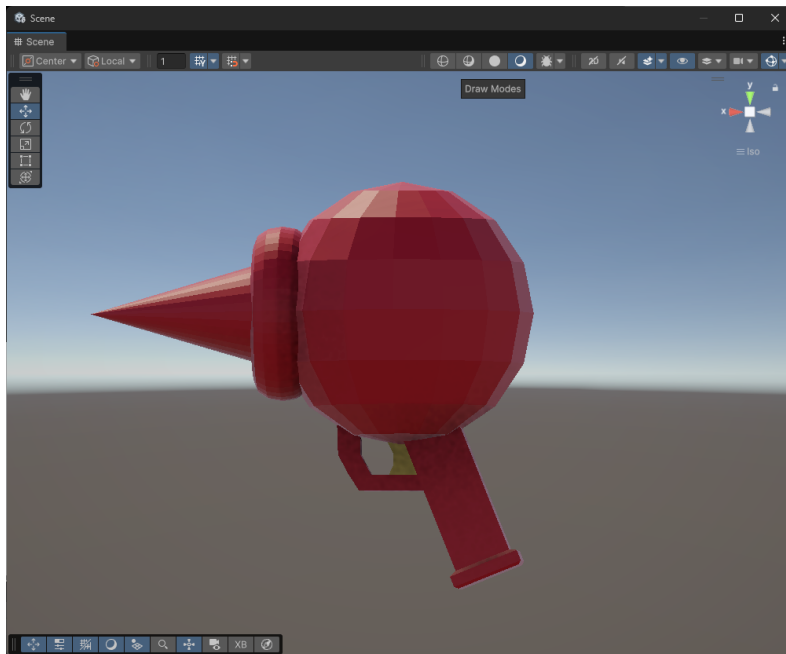
3.2 掴めるオブジェクトの最小構成

オブジェクトを掴めるようにするためには、以下の 4 つの要素が必須となります。

1. Rigidbody：物理挙動を制御します。VR で掴む対象には、勝手に落下しないよう Is Kinematic を状況に応じて使い分けます。
2. Collider：当たり判定です。掴む対象の形状に合わせて適切な Collider を選択します。
3. Grabbable：掴むロジックのコアです。
4. Grab Interactable：掴むための具体的な設定を行うコンポーネントです。

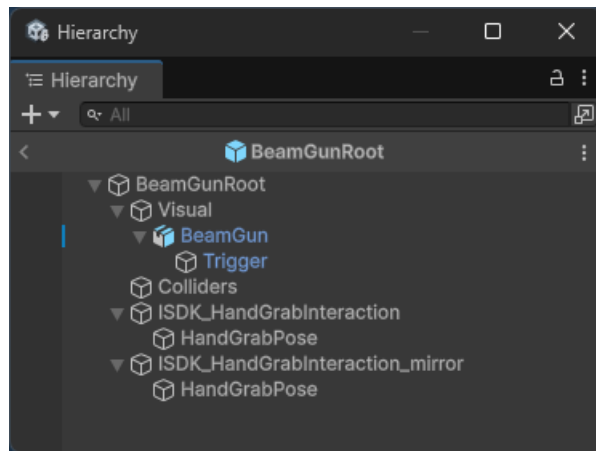
3.3 実践：ビームガンの実装手順

サンプルゲームの主役であるビームガン为例に、掴む動作の実装手順を解説します。あなたが作りたいオブジェクトに合わせて、適宜調整してみてください。



▲図 3.1 ビームガンのモデル

はじめに、完成したビームガンの構成は図 3.2 のようになります。



▲図 3.2 ビームガンの Hierarchy

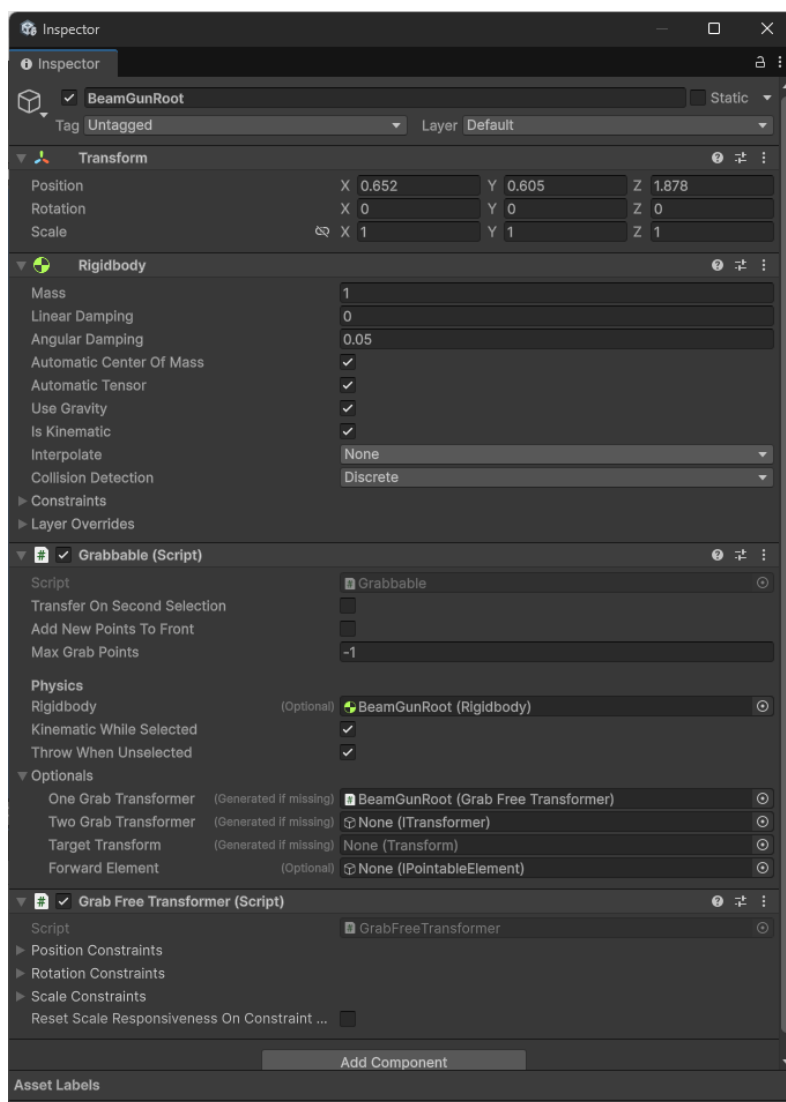
要素の説明は以下の通りです。

- Root：ビームガン全体の親オブジェクト
 - Visual：3D モデル
 - Colliders：当たり判定用
 - ISDK_GrabInteraction：ISDK の Grab Interactable コンポーネントを設定

それでは、順を追って実装していきましょう。まずは Empty Object を作成して、Root とします。Root には Rigidbody、Grabbable、Grab Free Transformer の 3 つのコンポーネントを追加します。

図 3.3 を参考に、各コンポーネントの設定も行います。Rigidbody の Is Kinematic を有効にしておくと、勝手に落下することがなくなります。Grabbable の Rigidbody には Root の Rigidbody を指定しておきましょう。また、One Grab Transformer には Root の Grab Free Transformer を指定しておきます。

ここで Grab Transformer とは、掴んだオブジェクトの位置や回転をどのように制御するかを定義するコンポーネントです。今回追加した Grab Free Transformer は、Interactable が Interactor に完全に追従するような挙動を実現します。この設定を、Grabbable の One Grab Transformer（片手で掴む場合の挙動）に指定することで、ビームガン片手で掴んだ際、完全に追従するようになります。

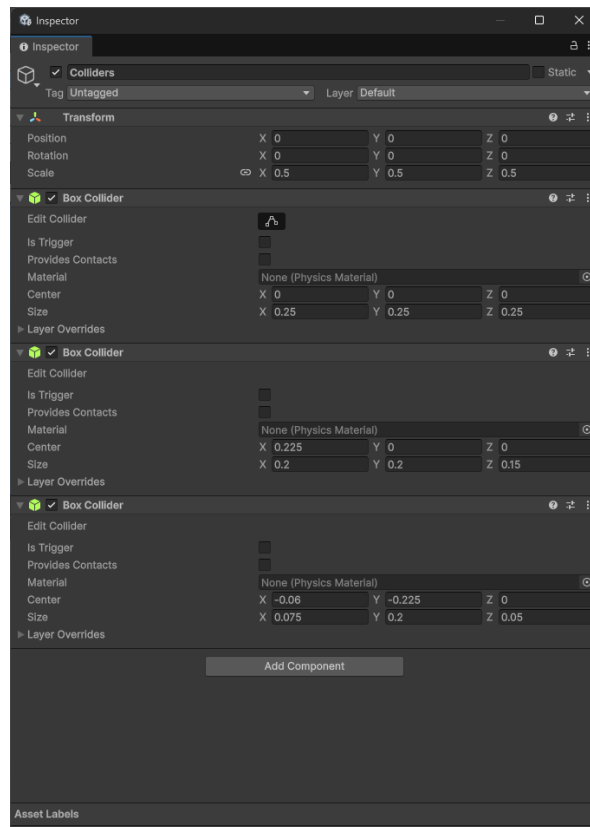


▲図 3.3 Root のコンポーネント

つづいて、Root に空の子オブジェクトを 2 つ追加し、Visual、Colliders とします。この Visual には特別追加するコンポーネントはありません。子要素として、ビームガンの 3D モデルを配置しておきましょう。

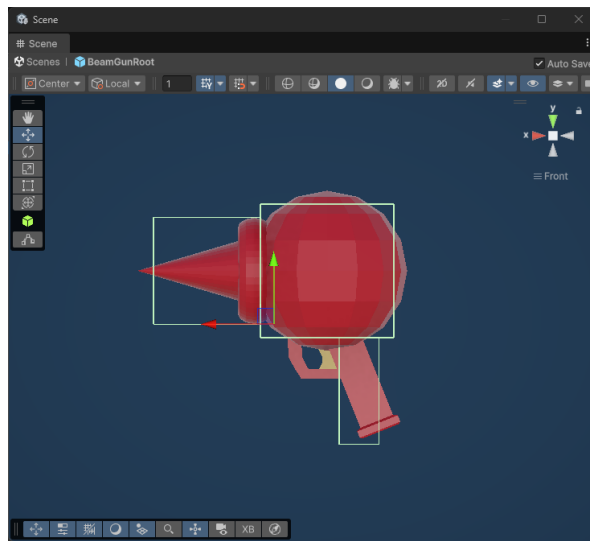
Colliders には、ビームガンの当たり判定用の Collider を配置します。図 3.4 のように、Colliders 自身に複数の Collider を追加してもよいですし、子要素に複数の Collider 用

オブジェクト を配置しても構いません。



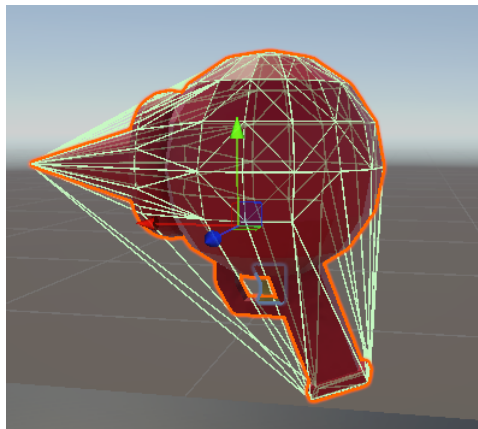
▲図 3.4 Colliders の構成

図 3.5 のようになるべく単純な形状の Collider を配置することで、物理挙動が安定しやすくなります。



▲ 図 3.5 Collider の配置例

または図 3.6 のように、単一の Mesh Collider を追加して Convex を有効にすることで、簡単に複雑な形状の当たり判定を作成することもできます。

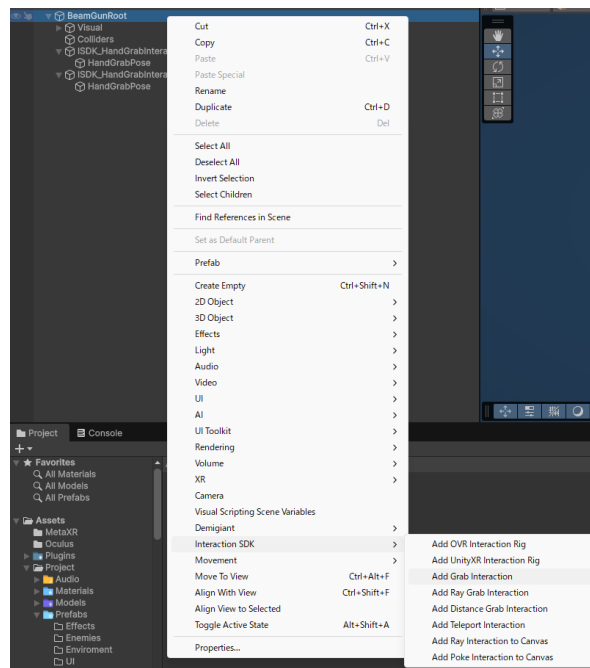


▲ 図 3.6 Mesh Collider の例

最後に Root に ISDK_GrabInteraction を追加します。

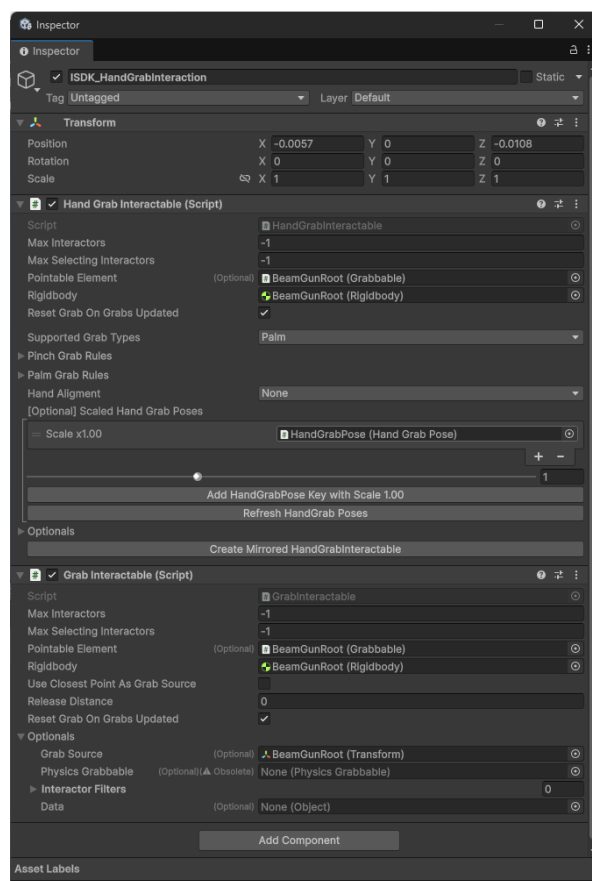
Root オブジェクトを右クリック > Interaction SDK > Add Grab Interaction で追加

できます。



▲図 3.7 Grab Interaction の追加

図 3.8 のように、Pointable Element、Rigidbody、Grab Source にはそれぞれ Root オブジェクトを参照しておきます。

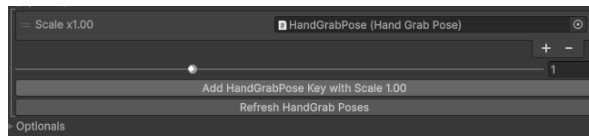


▲図 3.8 Grab Interaction の設定

これにて最小限の構成は完了です。XR Simulator や 実機ビルドをして、実際に掴む動作をテストしてみましょう。

3.4 ハンドポーズとミラーリング

前節で実装した掴む動作をより自然にするには、ハンドポーズの設定が必要不可欠です。ISDK_HandGrabInteraction の Hand Grab Interactable コンポーネントにある、Add HandGrabPose Key with Scale 1.00 をクリックして、ハンドポーズを追加してみましょう。



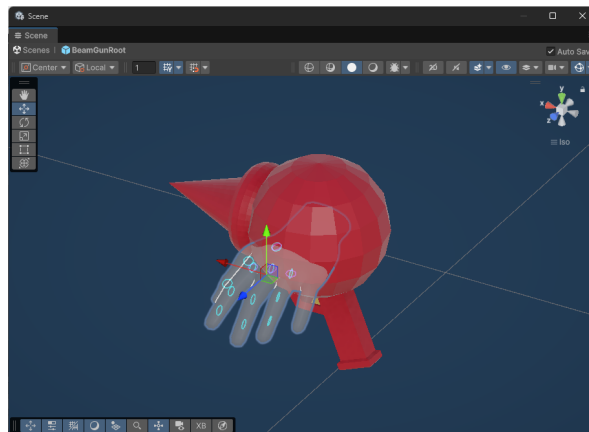
▲図 3.9 ハンドポーズの追加

すると ISDK_HandGrabInteraction の子要素に HandPose が追加されます。



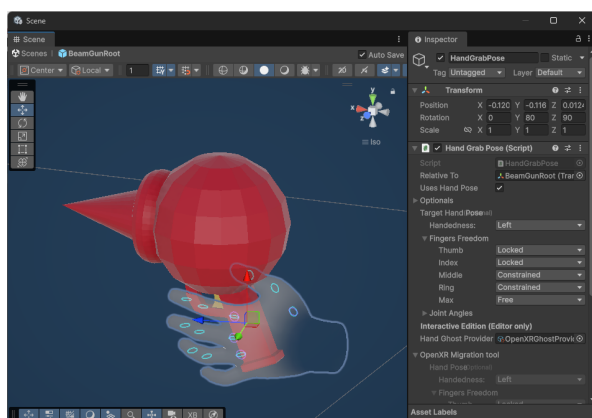
▲図 3.10 HandPose の追加

初期ポーズは図 3.11 のような、手を開いた状態になっています。



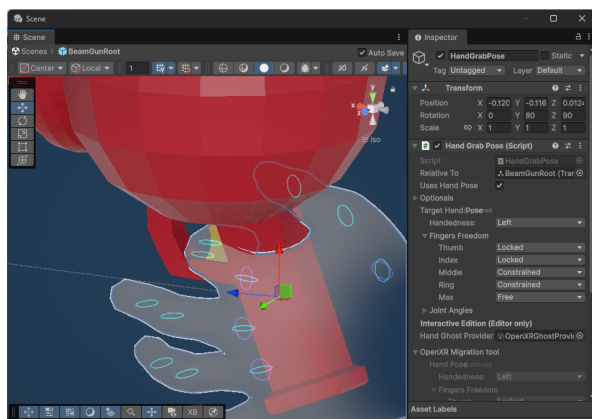
▲図 3.11 初期ポーズ

ビームガンを掴んでいるハンドポーズを想像しながら、まずは Transform を調整して、大まかな位置と回転を合わせましょう。



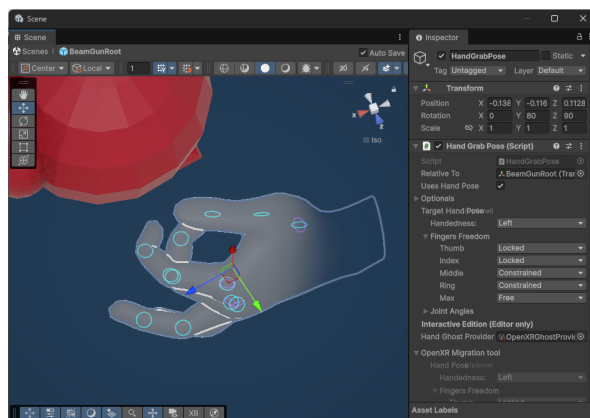
▲図 3.12 大まかな調整

シーン上で、ハンドポーズオブジェクトの各関節部分にあるリングをマウスで回転させることで、関節を回転させることができます。



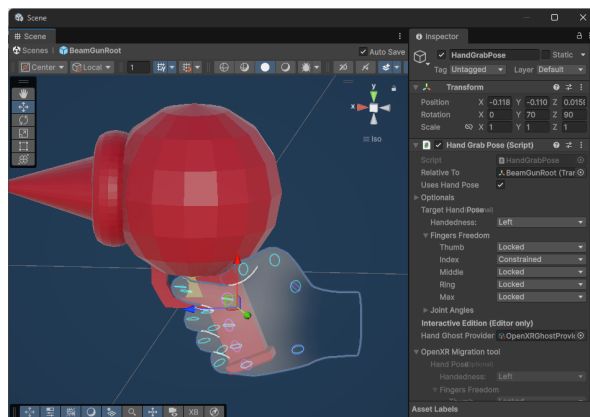
▲図 3.13 関節の調整

オブジェクトと手が重なって見えづらい場合は、適宜ハンドポーズオブジェクトをスライドして調整してもいいでしょう。



▲図 3.14 ハンドポーズオブジェクトのスライド

各指を調整して、自分の想像したハンドポーズになれば完成です。今回のビームガンを持つハンドポーズは、図 3.15 のようになりました。



▲図 3.15 ビームガンを持つハンドポーズ

ハンドポーズを調整するコツ

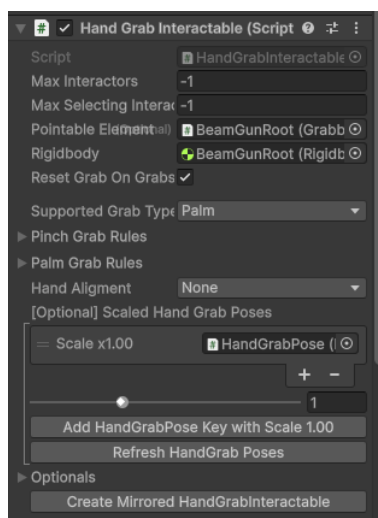
ハンドポーズの調整は手作業で行うため、感覚を頼りにする部分が大きいです。

調整のコツとして以下があげられます。

- 現実で自分の手や資料を参考にしながら、模写のように調整する。
- はじめはおおまかな手の位置を合わせたり、根本の関節から動かしてみるなど、大きな動きのある部分から調整する。

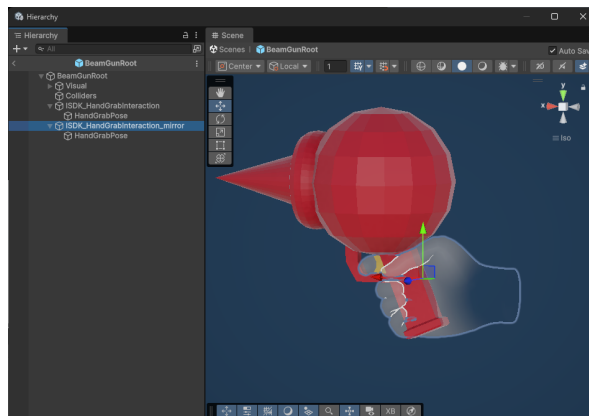
はじめから細部にこだわりすぎず、まずは全体のバランスを見ながら作業を進めてみましょう。

完成したハンドポーズは、反対の手にもミラーリングして適用することができます。ISDK_HandGrabInteraction の Hand Grab Interactable コンポーネントにある、Create Mirrored HandGrabInteractable を押してみましょう。



▲図 3.16 ミラーリングの実行

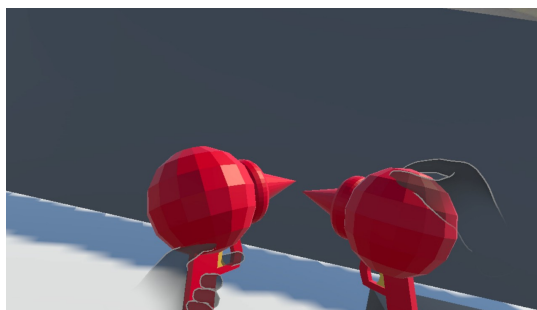
すると Root オブジェクトの子要素に、ISDK のミラーが追加されます。これで、面倒な調整をスキップして反対側の手についてもハンドポーズが作成できました。



▲図 3.17 ミラーオブジェクト

ハンドポーズを設定した場合//設定しない場合の違い

ハンドポーズを設定の有無で、掴む動作について比較を行った。(図 3.18) 左がハンドポーズを設定したオブジェクトで、右がハンドポーズを設定していないオブジェクトを掴んでいる様子である。左はコライダーの範囲内でジェスチャーをすると、手が自然にオブジェクトにフィットしているのに対し、右はコライダーがついているところならどこでも自由に掴めてしまう。



▲図 3.18 ハンドポーズの有無の比較

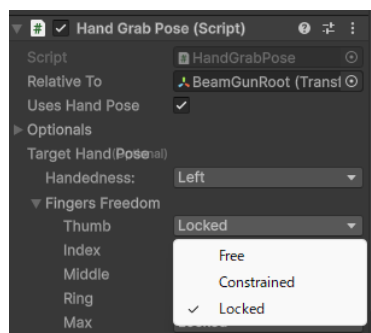
3.5 掴み方のルールと挙動の設定

ISDK では、掴み方のスタイルを詳細に設定できます。

3.5.1 Fingers Freedom

Hand Grab Pose の Fingers Freedom では、各指に対して自由度を設定できます。これはハンドポーズをとった際、指がどの程度自由に動くかを定義するもので、以下の3つのモードがあります。

- Free：指は自由に動きます。
- Constrained：指はポーズの位置を超えて動くことはありません。
- Locked：指はポーズの位置に完全に固定されます。

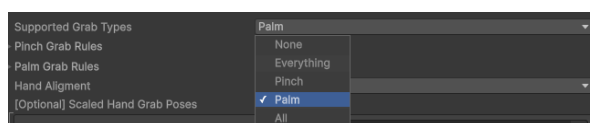


▲図 3.19 Fingers Freedom の設定

3.5.2 Supported Grab Types

Hand Grab Interactable の Supported Grab Types には、どの掴み方に対応するかを設定できます。

- Pinch：親指と人差し指をくっつけるような掴み方で、例えばチェスの駒をつまむときに適しています。
- Palm：手のひら全体で掴むスタイルで、今回のビームガンのようなオブジェクトを掴むときに適しています。



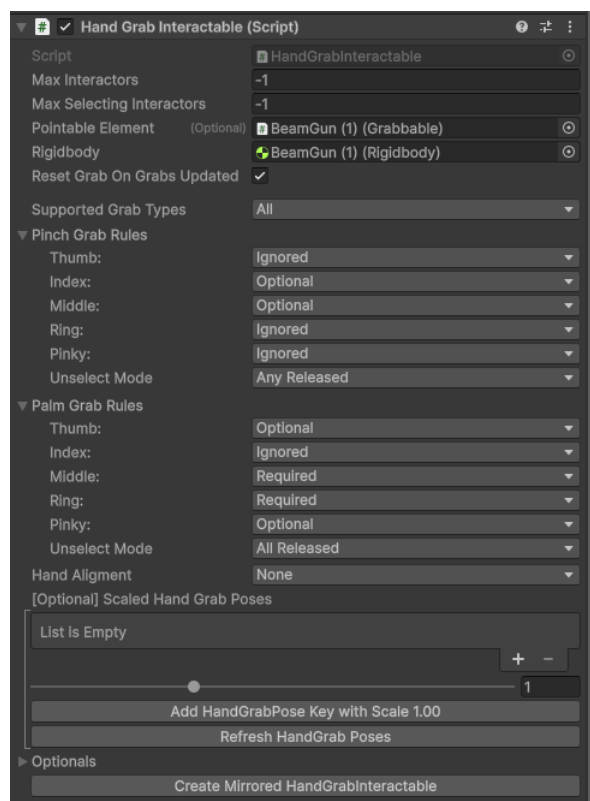
▲図 3.20 Supported Grab Types の設定

3.5.3 Palm Grab Rules

Hand Grab Interactable の Palm Grab Rules では、どの指がポーズ（ジェスチャー）と一致していたらグラブをトリガーするか決定できます。項目は以下のいずれかです。

- Required（必須）
- Optional（任意）
- Ignored（無視）

グラブ動作がトリガーされるには、すべての Required な指がジェスチャーを実行しなければなりません。Required マークが一つもない場合、少なくとも一つの Optional な指がジェスチャーを実行しなければなりません。Ignored な指は、グラブ動作のトリガーにおいて完全に無視されます。

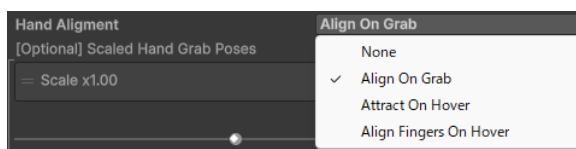


▲図 3.21 Palm Grab Rules の設定

3.5.4 Hand Alignment

Hand Grab Interactable の Hand Alignment の項目は、画面上の手がインタラクトに合わせてどのように動くかを設定できます。

- AlignOnGrab：この動作は、グラブ開始時点で手を Interactable に視覚的に自動スナップさせます。これがデフォルトの動作です。
- AttractOnHover：これは、ホバリングが強くなるにつれて、最終的なハンドポーズを段階的に適用します。
- AlignFingersOnHover：これは、ホバリング中にユーザーが手を閉じ始めると、指の最終ポーズを段階的に適用します。
- None：画面上の手にハンドポーズは視覚的に適用されません。



▲図 3.22 Hand Alignment の設定

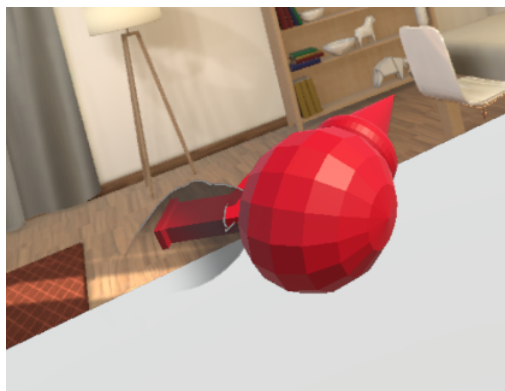
3.6 Meta XR Simulator で確認する

第2章で解説した Meta XR Simulator を活用して、ビームガンを持つ動作の検証をしましょう。XR Simulator を有効にして、シーンを再生してください。

キーボードの2または4で掴み、3でつまむことができます。(左クリックまたはキーボードの1で解除) 前述した、Supported Grab Types が Palm だと、3を押してもつまむことができません。

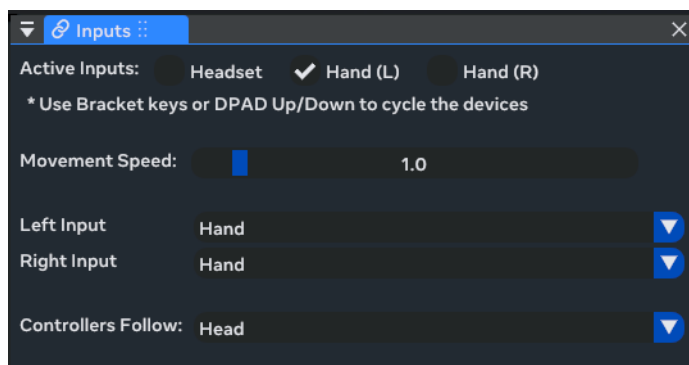


▲図 3.23 Pinch 動作では掴めないビームガンの様子



▲図 3.24 Palm 動作でビームガンを持っている様子

Inputs の Active Inputs を Hand(L) にすることで、左手のみの精密な操作に切り替えられます。また、Left Input や Right Input が Hand ではなく Controller になっていると、Hand Grab Interactable をつけたオブジェクトは反応しないので注意しましょう。



▲図 3.25 Active Inputs の画面

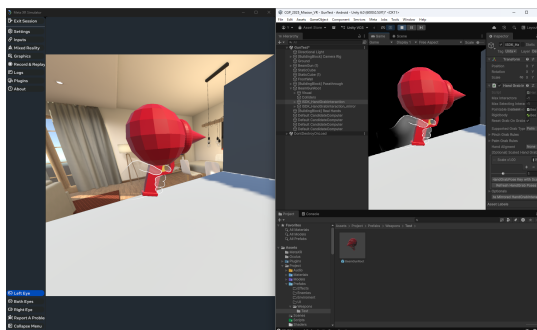
この状態で手の姿勢を操作して、最終的な見栄えを確認しましょう。



▲図 3.26 最終的な姿勢

掴んでいないのにハンドポーズが表示される

XR Simulator で確認した際、掴んでいないのに図 3.27 のようにシミュレーション画面にハンドポーズが表示されることがあります。この原因は、Hierarchy 上で ISDK_HandGrabInteraction を選択していることです。理想的な状態のハンドポーズを確認する手段として使ってみてもいいでしょう。



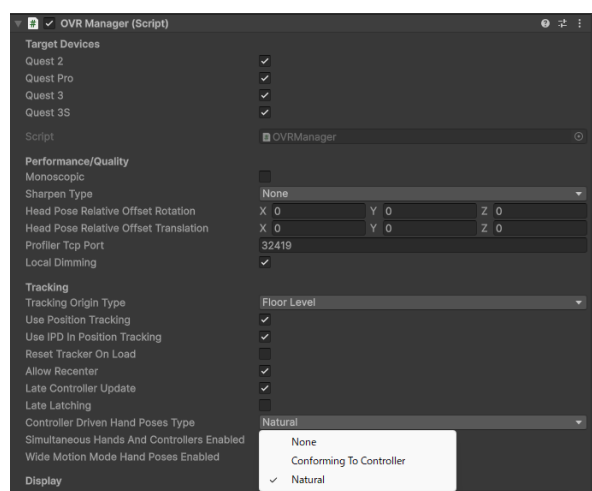
▲図 3.27 ISDK_HandGrabInteraction を選択している状態

3.7 手とコントローラーの違い

Meta Quest 3 では、手のトラッキングとコントローラーの両方を使用してインタラクションが可能です。両者の性質は、入力方法（手のジェスチャー vs ボタン操作）や、触覚フィードバックの有無、精度など、様々な点で異なります。ISDK では、これらの違いを考慮し、「Hand Grab」と「(Controller) Grab」の別々の Interactable として、同じオブジェクトに持たせることができます。これにより、ユーザーは手でもコントローラーでも自然にオブジェクトを掴むことができるようになります。

ところで、コントローラーを使用しながらも、コントローラーがどのような振る舞いをするかを設定できる項目が存在します。

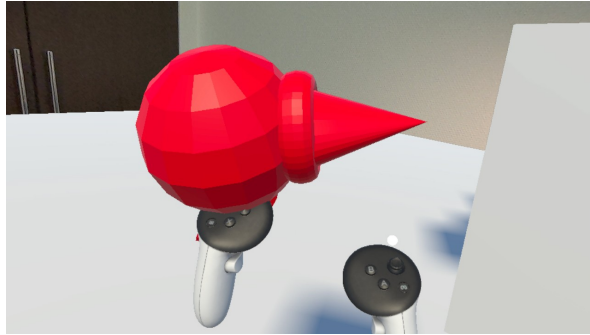
Building Block で追加した Camera Rig についている OVR Manager を見てみましょう。OVR Manager の項目の中に、Controller Driven Hand Poses Type という項目があります。



▲図 3.28 Controller Driven Hand Poses Type

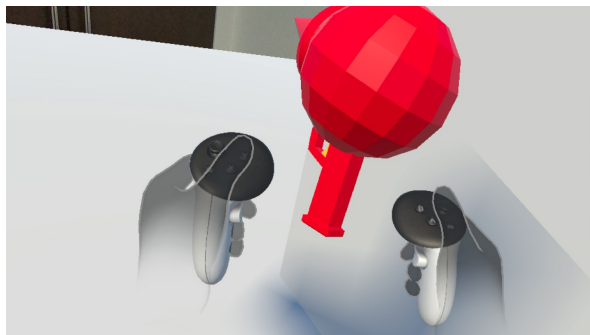
この項目を設定することで、Controller がどのように駆動するかを設定できます。

- None：常にコントローラーが表示される。掴み判定はコントローラー（Grab Interactable）

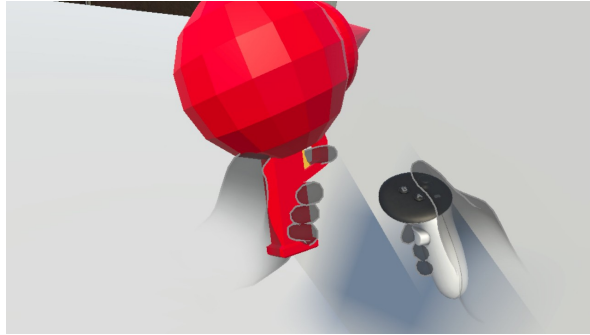


▲図 3.29 None

- Conforming to Controller：通常時は図 3.30 のように、コントローラーを持った手が表示される。物を掴むと図 3.31 のように手だけになる。掴み判定は手（Hand Grab Interactable）

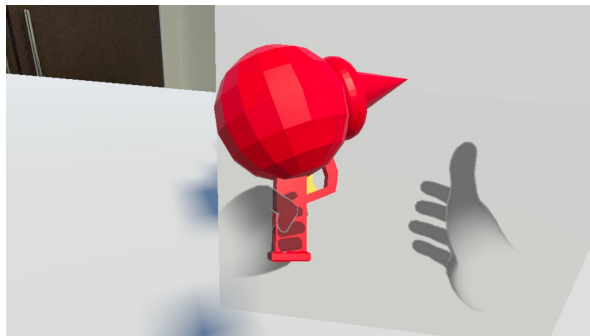


▲図 3.30 Conforming to Controller（通常時）



▲図 3.31 Conforming to Controller (掴んだとき)

- Natural：常に手が表示される。掴み判定は手 (Hand Grab Interactable)



▲図 3.32 Natural

このように、コントローラーで掴んだ際の判定をコントローラーにするか、手にするかを選択できます。この資料では Conforming to Controller を使い、Hand Grab Interactable をコントローラーでも掴めるようにします。

第 4 章

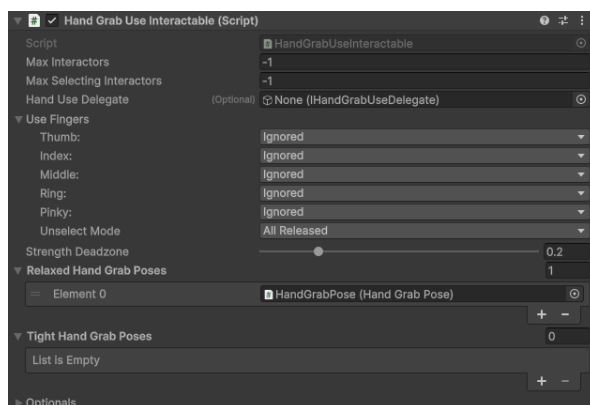
ゲーム性の実装 ーメカニクスとイベント

第 3 章ではオブジェクトを掴むための基本的な設定を学びました。この章では、掴んだオブジェクトの「トリガーを引く」「ボタンを押す」といった、二次的な操作（Use）と、それに連動するイベントの設定方法を解説します。

4.1 「使う」動作の設定

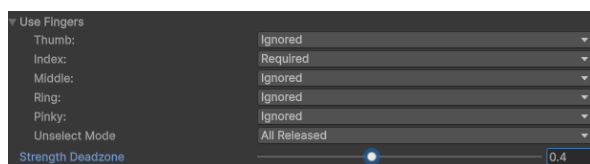
ISDK には、「掴む（Grab）」とは別に、掴んでいる最中にトリガーを指で引き込む動作を検知する Hand Grab Use という仕組みがあります。

この動作を実装するために、Root の子要素である ISDK オブジェクトに、Hand Grab Use Interactable コンポーネントを追加します。



▲図 4.1 Hand Grab Use Interactable の追加

まずは Use Fingers を設定していきます。今回、ビームガンのトリガーを引く動作を検知したいので、Index（人差し指）を Required に設定しておきましょう。また、Strength Deadzone を調整することで、どの程度の押し込みでトリガーを発動させるか設定できるため、0.4 程度にしておきましょう。



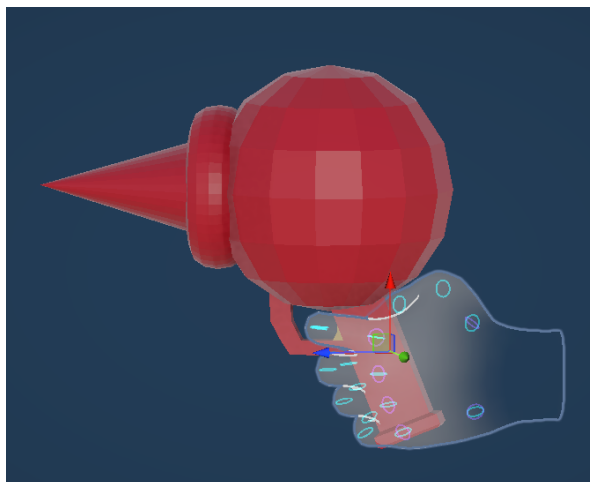
▲図 4.2 Use Fingers の設定

また、新たなハンドポーズとして、トリガーを引いた時のポーズを作ります。もともとあった HandGrabPose を複製して、わかりやすいように TriggeredHandGrabPose と名前を変更しておきましょう。



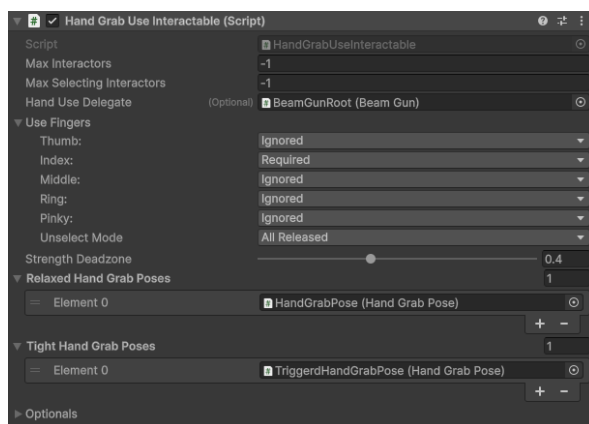
▲図 4.3 TriggeredHandGrabPose の用意

もとのポーズから、人差し指の関節を少し曲げる程度で調整してみましょう。



▲図 4.4 TriggeredHandGrabPose の調整

Hand Grab Use Interactable の Relaxed Hand Grab Pose に、もともと存在した HandGrabPose を、Tight Hand Grab Pose に先ほど調整した Triggered Hand Grab Pose を指定します。

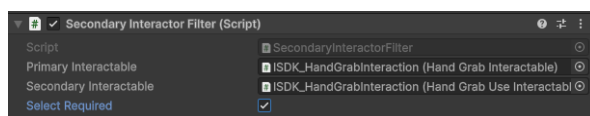


▲図 4.5 Relaxed Hand Grab Pose と Tight Hand Grab Pose の設定

次に、同じく ISDK オブジェクトに、Secondary Interactor Filter コンポーネントを追加します。そして Primary Interactable にこのオブジェクトの Hand Grab Interactable を指定し、Secondary Interactable に Hand Grab Use Interactable を指定します。これ

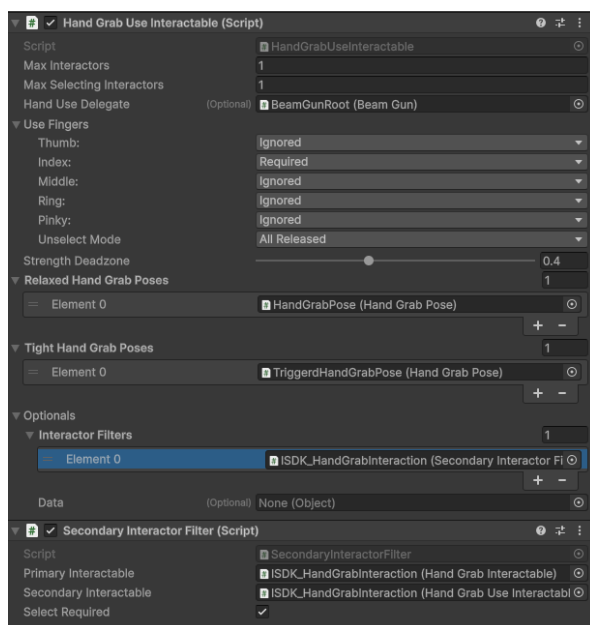
第4章 ゲーム性の実装 —メカニクスとイベント

により、掴むインタラクションを1次、使うインタラクションを2次に設定できました。さらに Select Required を有効にします。これにより、デフォルトでは1次インタラティブ要素のホバー時に、2次インタラクションが可能ですが、Select Required を有効にした影響で、1次要素の選択時にのみ（掴んでいる時のみ）2次インタラクションが可能になります。



▲図 4.6 Secondary Interactor Filter の設定

設定が完了したら、Hand Grab Use Interactable の Options を開き、Interactor Filters の項目に、先ほど設定した Secondary Interactor Filter を追加しておきましょう。



▲図 4.7 Hand Grab Use Interactable の Options に追加

これで、使う動作の設定は完了です。

4.2 「使う」動作の検知

ではこの「使う」動作を検知し、ビームガンのトリガーを引いたときに音を鳴らしてみよう。

BeamGun.cs というスクリプトを新規作成し、以下のコードを記述します。

▼リスト 4.1 BeamGun.cs のコード

```
1: using UnityEngine;
2: using Oculus.Interaction;
3:
4: public class BeamGun : MonoBehaviour, IHandGrabUseDelegate
5: {
6:     [SerializeField]
7:     private AudioSource _audioSource;
8:     [SerializeField]
9:     private AudioClip _shootSound;
10:
11:     public void BeginUse()
12:     {
13:         _audioSource.PlayOneShot(_shootSound);
14:     }
15:
16:     public void EndUse()
17:     {
18:     }
19:
20:     public float ComputeUseStrength(float strength)
21:     {
22:         return strength;
23:     }
24: }
```

このコードで重要なのは、IHandGrabUseDelegate インターフェースを実装している点です。これにより、Hand Grab Use Interactable のイベントを受け取ることができます。

BeginUse メソッドは、トリガーを引く動作が開始されたときに呼び出されるため、ここで音を鳴らす処理を記述しています。

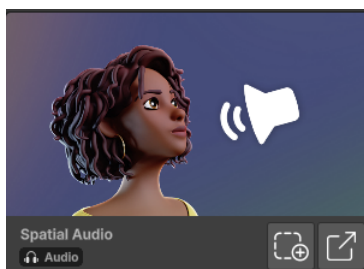
EndUse メソッドは、トリガーを引く動作が終了したときに呼び出されますが、今回は特に処理は必要ないため空のままにしています。

ComputeUseStrength メソッドは、トリガーの引き具合に応じた強さを計算するためのもので、今回はそのままの値を返すようにしています。

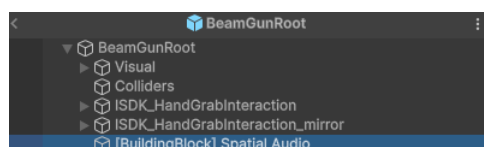
このスクリプトは、ビームガンの Root オブジェクトに追加しておきましょう。

第4章 ゲーム性の実装 —メカニクスとイベント

ここで、一つ注意点があります。通常のカメラなら、AudioSource コンポーネントから音が聞こえますが、XR 環境では Spatial Audio（空間オーディオ）が必要になります。そのため、Building Blocks から Spatial Audio（図 4.8）をシーンに追加し、Root オブジェクトの子要素にしておきましょう。Spatial Audio の詳細については、次の章で説明します。

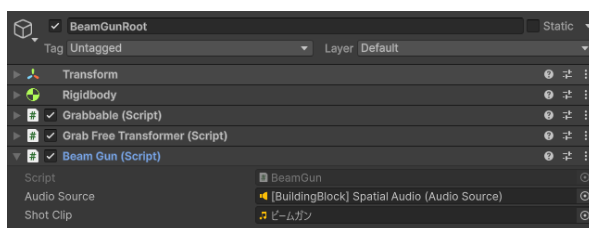


▲図 4.8 Spatial Audio



▲図 4.9 Root の子要素に Spatial Audio を配置する

ここまでできたら、Root に追加した BeamGun スクリプトに、必要な要素を参照しましょう。ビームガンの効果音は、フリーの音源などから適当に探してきます。



▲図 4.10 Root に追加した BeamGun スクリプト

これで準備は完了です。実機ビルドをして、ビームガンのトリガーを引いたときに音が

鳴るか確認してみましょう。

トリガーの引き具合を見た目に反映する

ここからは少々応用的な内容になりますが、トリガーの引き具合に応じて、ビームガン見た目を変化させてみましょう。前提として、ビームガンなどの 3D モデルのトリガー部分を、シェイプキーで動かせるようにしておく必要があります。

まず、BeamGun.cs で実装した IHandGrabUseDelegate の ComputeUseStrength メソッドを以下のように変更します。

▼リスト 4.2 ComputeUseStrength の変更

```
1: private float _useStrength;  
2:  
3: public float ComputeUseStrength(float strength)  
4: {  
5:     _useStrength = strength;  
6:     return strength;  
7: }
```

このように、引数で受け取った strength をクラスのフィールドに保存するようにします。次に、この _useStrength を受け取って、トリガーの見た目を変える処理を追加します。

▼リスト 4.3 トリガーの見た目を変える処理の追加

```
1: [SerializeField]  
2: private SkinnedMeshRenderer _triggerRenderer;  
3:  
4: public void UpdateTriggerVisual(float strength)  
5: {  
6:     _triggerRenderer.SetBlendShapeWeight(0, strength * 100f);  
7: }
```

あとは _useStrength の変更を何らかの方法で検知し、UpdateTriggerVisual メソッドを呼び出せば OK です。

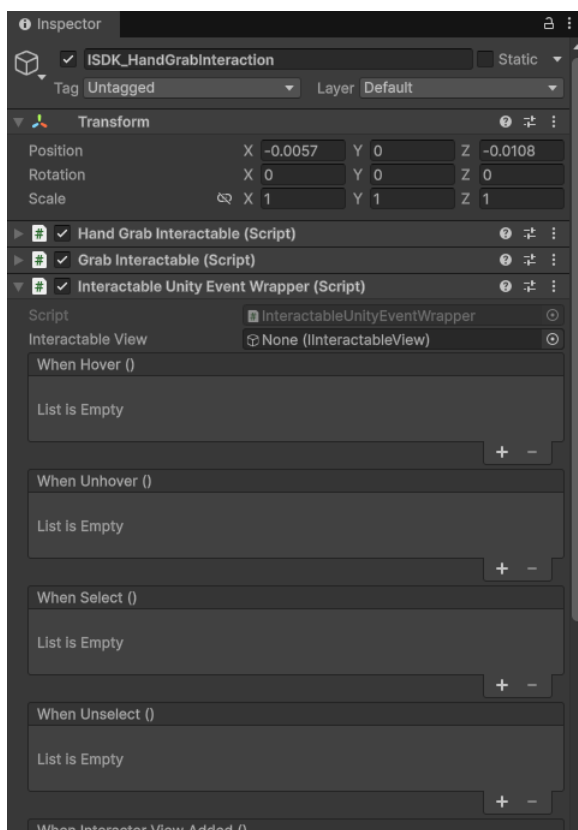
4.3 発展編

ここからは、ビームガンの実装自体とは関係ありませんが、ISDK の機能について、より深く解説していきます。

4.3.1 Interactable Unity Event Wrapper

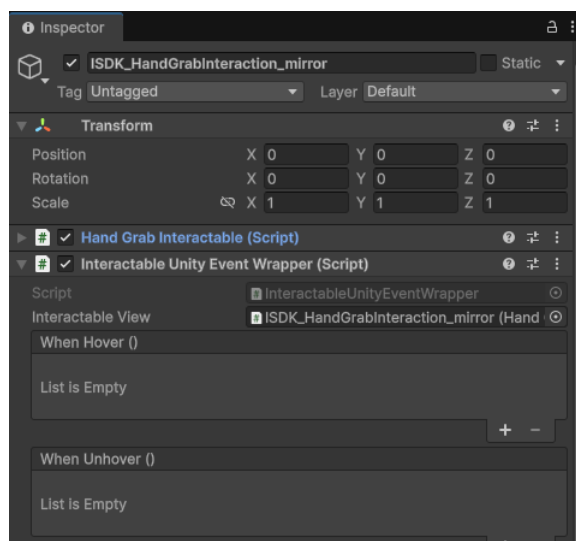
ISDK には、インタラクションの変化を検知してイベントを発火させる Interactable Unity Event Wrapper というコンポーネントがあります。これを使えば、例えば掴んだ時、手放した時に特定の処理を呼び出すといったことが簡単に実装できます。

Interactable Unity Event Wrapper を ISDK オブジェクトに追加してみましょう。



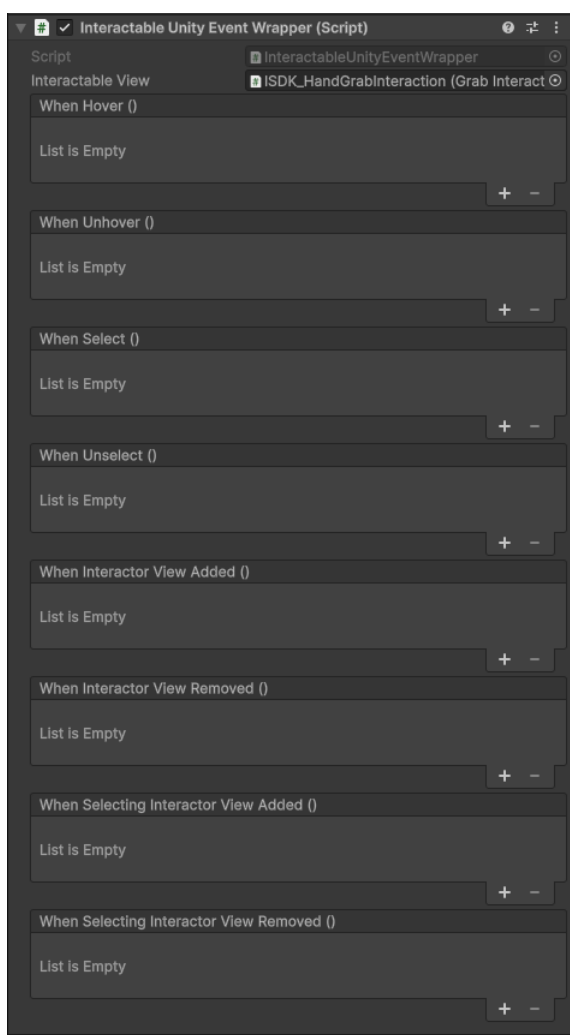
▲図 4.11 Interactable Unity Event Wrapper の追加

Interactive View には、どの Grab Interactive を対象にするか設定します。今回は、ISDK オブジェクトについている Hand Grab Interactive を指定しておきましょう。なお、反対側の手にも同じ機能を実装したい場合、ミラーオブジェクトにも同様にこのコンポーネントを追加しましょう。



▲図 4.12 Interactive View の設定

このコンポーネントは通常の UnityEvent を購読するのと同じように、インスペクター上でイベントを追加できます。例えば BeamGun.cs に装備音を鳴らすメソッドなどを用意し、When Select イベントに追加すると、掴んだときに装備音が鳴るようになります。

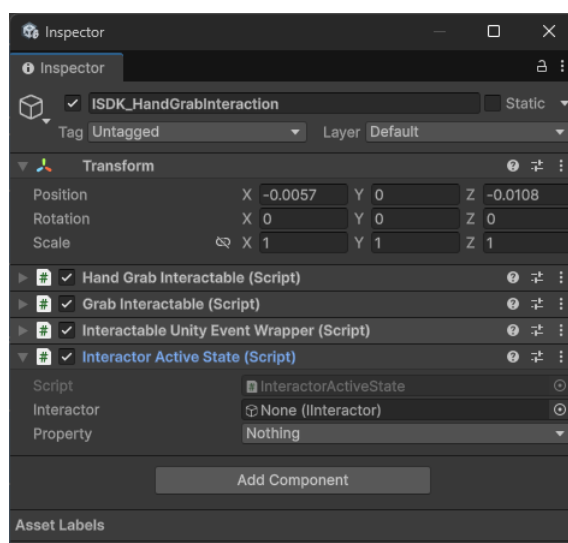


▲図 4.13 イベント購読

4.3.2 Interactor の状態検知

この節では、コントローラーが何かを掴んでいる、など、Interactor 自体の状態を検知する手法を説明します。

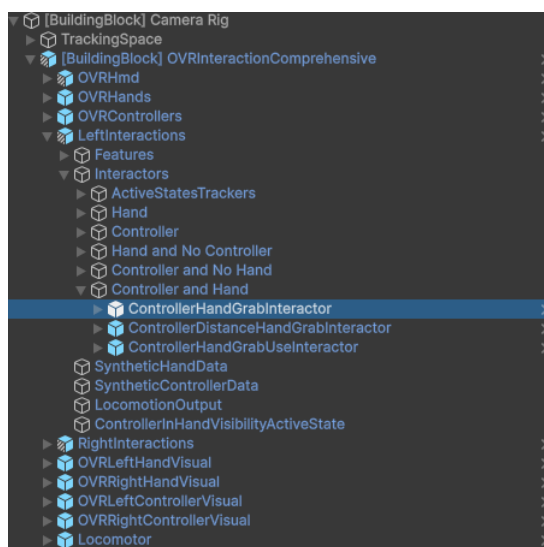
ISDK オブジェクトに、Interactor Active State コンポーネントを追加します。



▲図 4.14 Interactor Active State の追加

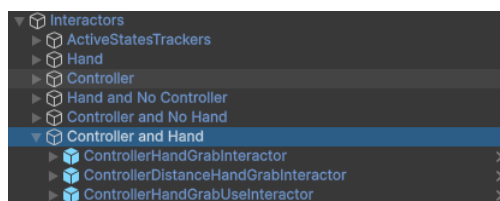
Interactor フィールドには、状態を検知したい Interactor を指定します。

CameraRig > OVRInteractionComprehensive > Left(Right)Interactions > Interactors のいずれかの Interactor を指定してみましょう。



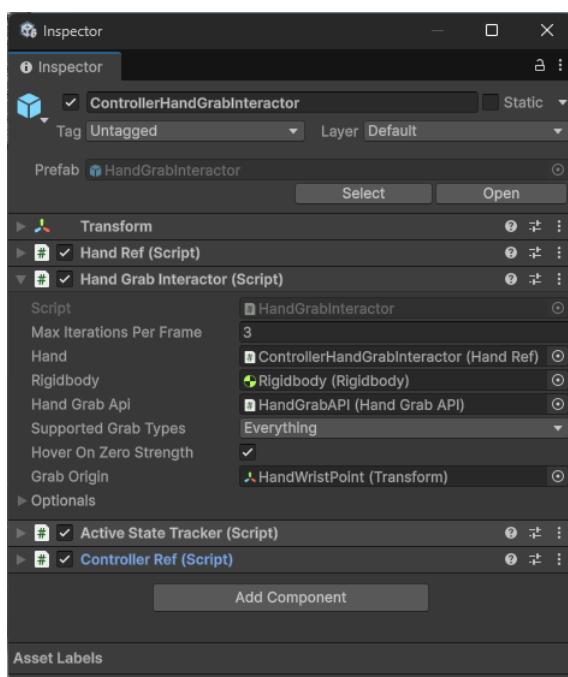
▲図 4.15 Interactor の指定

例えば、Controller and Hand には 3 つの子要素があり、



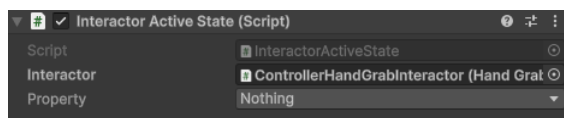
▲図 4.16 Controller and Hand の子要素

ControllerHandGrabInteractor には Hand Grab Interactor がついています。



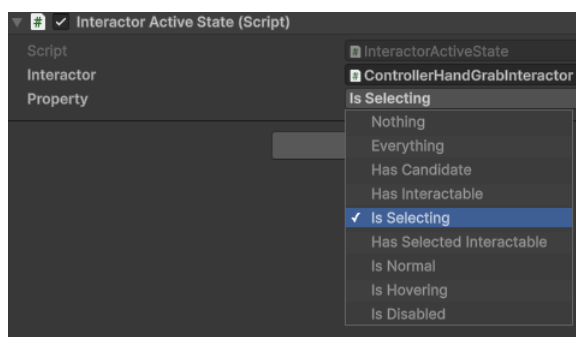
▲図 4.17 ControllerHandGrabInteractor の Inspector

これを先ほどの ISDK オブジェクトにつけた InteractorActiveState の Interactor フィールドに指定してみましょう。



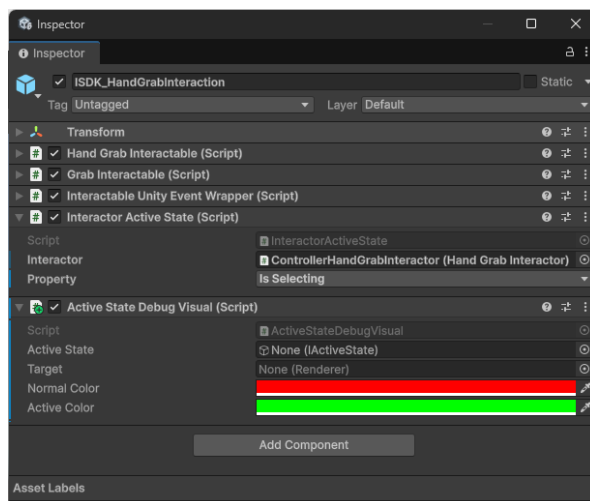
▲図 4.18 ControllerHandGrabInteractor を指定

Interactor が設定出来たら、どの状態を検知するか Property から設定します。例えば、掴んでいる状態を検知する場合は Is Selecting を選択しましょう。なお、検知する状態は複数選択可能です。



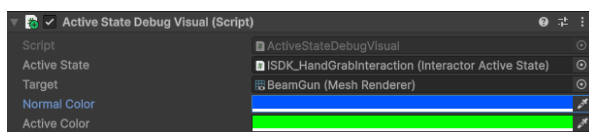
▲図 4.19 Property の設定

次に、この `InteractorActiveState` を参照して、実際に掴んだ状態を検知できているか確かめてみます。ISDK にはデバッグ用に、Active State Debug Visual というコンポーネントがあります。これを ISDK オブジェクトに追加しましょう。



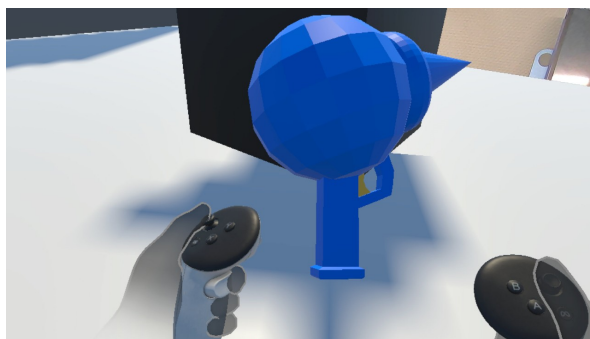
▲図 4.20 Active State Debug Visual の追加

Active State には `Interactor Active State` を指定します。Target はこのオブジェクトの `Renderer` を指定しておきましょう。カラーについても好みで設定してみてください。

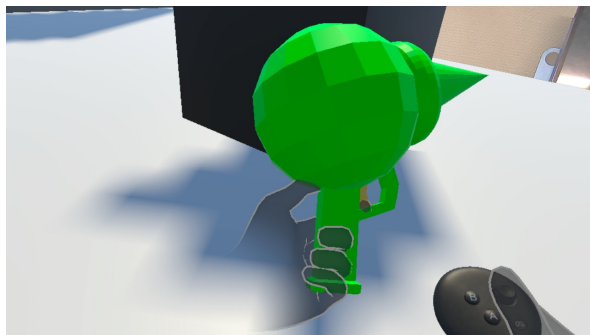


▲図 4.21 Active State Debug Visual の設定

これでデバッグの準備は完了です。以下に実際に実機でデバッグしたときの画像を掲載しています。



▲図 4.22 通常時

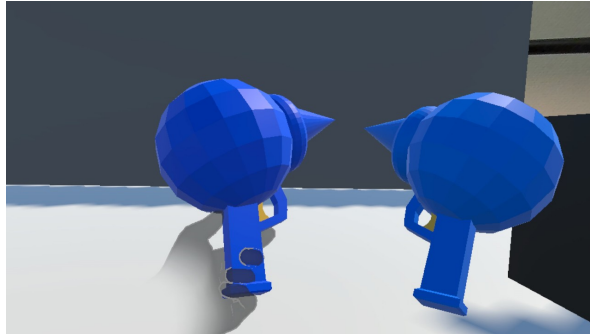


▲図 4.23 掴んでいる状態

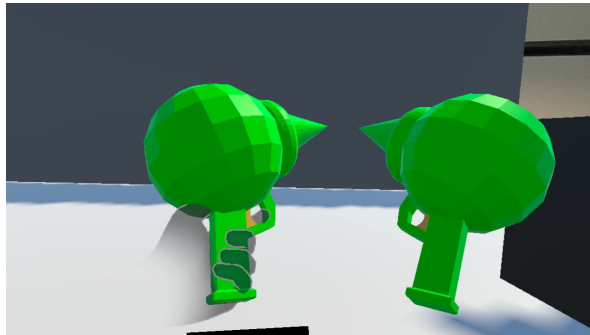
確かに状態を検知していることが確認できました。
もし正しく動作しなかった場合は、参照が間違っていないか、設定した方の手で掴んでい

るかを確認しましょう。

なお、この方法はあくまで Interactor を参照しているため、どの Interactable を掴もうが、同じ Interactor を参照している限りは反応してしまうので注意してください。

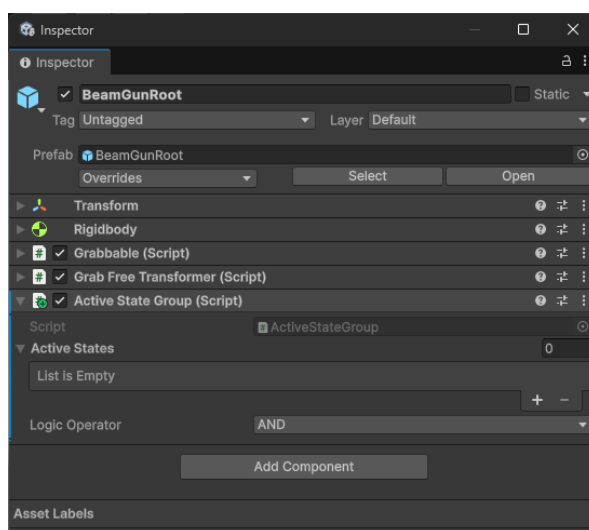


▲ 図 4.24 何も掴んでいないとき



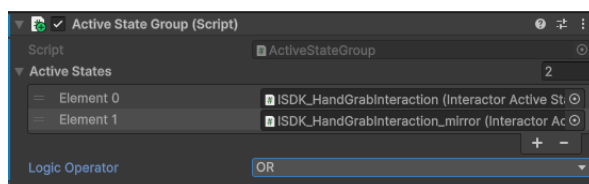
▲ 図 4.25 片方のオブジェクトを掴むと、もう片方も反応する

片方の手だけでなく、両手のアクティブ状態をまとめて検知するには、Active State Group コンポーネントを使用する。手順は省略するが、反対側の ISDK オブジェクトにも同じように Interactor Active State を設定し、Root オブジェクトに ActiveStateGroup コンポーネントを追加する。



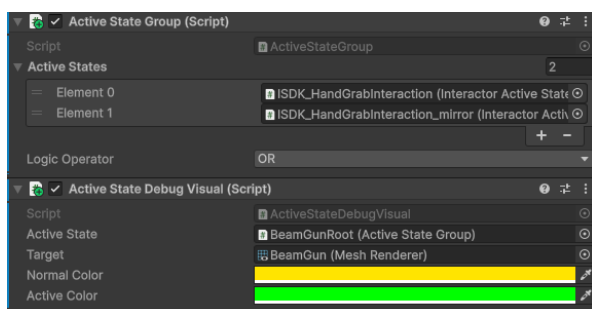
▲図 4.26 Active State Group の追加

Active States に 両方の ISDK オブジェクトの Interactor Active State を追加し、Logic Operator を設定します。AND ならどちらも満たした時、OR なら少なくともどちらかが満たした時、XOR ならどちらか一方が満たした時に、Active State Group がアクティブになります。



▲図 4.27 Active State Group の設定

これを先ほどのデバッグコンポーネントに参照させれば動作確認が可能です。



▲図 4.28 Active State Group を参照する

自作したスクリプトで `ActiveState` を参照する

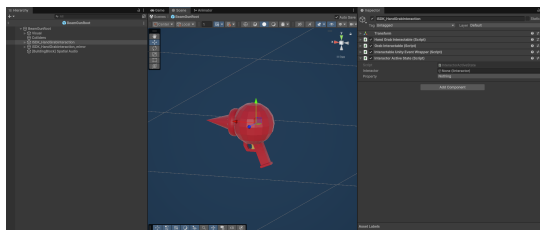
`Active State Debug Visual` コンポーネントではなく、実際に自作のスクリプトで `Active State` の `Active` 状態を検知する方法です。

▼リスト 4.4 Active State を参照するコード例

```
1: using UnityEngine;
2: using Oculus.Interaction;
3:
4: /// <summary>
5: /// コントローラーのActive状態(ActiveState)を取得して、
6: /// 状態に応じた処理を行うサンプル。
7: /// </summary>
8: public class ActiveStateSample : MonoBehaviour
9: {
10:     [SerializeField, Interface(typeof(IActiveState))]
11:     private Object _activeState;
12:     private IActiveState ActiveState { get; set; }
13:
14:     private void Start()
15:     {
16:         ActiveState = _activeState as IActiveState;
17:     }
18:
19:     private void Update()
20:     {
21:         bool isActive = ActiveState.Active;
22:
23:         if (isActive)
24:         {
25:             Debug.Log("Active");
26:         }
27:         else
28:         {
29:             Debug.Log("Inactive");
30:         }
31:     }
32: }
```

インスタンス化した時に Interactor を参照する方法

ISDK オブジェクトを含むオブジェクトがプレハブ化されていて、Instantiate して生成する場合、事前に Inspector 上で Interactor を参照しておくことができません。



▲図 4.29 プレハブの Inspector

この場合の対策として、`InteractorActiveState.cs` の最後に以下のようなコードがあります。

```
#region Inject

/// <summary>
/// Injects all required dependencies for a dynamically instantiated
/// <see cref="InteractorActiveState"/>.
/// This method exists to support Interaction SDK's dependency injection pattern and is not
/// needed for typical Unity Editor-based usage.
/// </summary>
public void InjectAllInteractorActiveState(IInteractor interactor)
{
    InjectInteractor(interactor);
}

/// <summary>
/// Sets the underlying <see cref="Interactor"/> for a dynamically instantiated
/// <see cref="InteractorActiveState"/>.
/// This method exists to support Interaction SDK's dependency injection pattern and is not
/// needed for typical Unity Editor-based usage.
/// </summary>
public void InjectInteractor(IInteractor interactor)
{
    _interactor = interactor as UnityEngine.Object;
    Interactor = interactor;
}

#endregion
```

▲図 4.30 `InteractorActiveState.cs` のコード（一部抜粋）

これらのメソッドを利用すれば、Inject する形で `Interactor` を参照することができます。詳細な手順は割愛しますが、状況に合わせて活用してみてください。

4.4 Interactable の状態検知

前節では `Interactor` の状態を検知する方法を説明しましたが、`Interactable` の状態を検知することも可能です。実装において特別なコンポーネントはなく、以下のようなコー

ドを書く必要があります。

▼リスト 4.5 Interactable の状態を検知するコード例

```

1: using Oculus.Interaction;
2: using UnityEngine;
3:
4: /// <summary>
5: /// Interactableの状態を取得して、状態に応じた処理を行うサンプル。
6: /// </summary>
7: public class InteractableStateSample : MonoBehaviour
8: {
9:     [SerializeField, Interface(typeof(IInteractableView))]
10:    private Object _interactableView;
11:    private IInteractableView InteractableView { get; set; }
12:
13:    private void Awake()
14:    {
15:        InteractableView = _interactableView as IInteractableView;
16:    }
17:
18:    private void OnEnable()
19:    {
20:        InteractableView.WhenStateChanged += HandleStateChanged;
21:    }
22:
23:    private void OnDisable()
24:    {
25:        InteractableView.WhenStateChanged -= HandleStateChanged;
26:    }
27:
28:    private void HandleStateChanged(InteractableStateChangeArgs args)
29:    {
30:        switch (InteractableView.State)
31:        {
32:            case InteractableState.Normal:
33:                Debug.Log("Normal");
34:                break;
35:            case InteractableState.Hover:
36:                Debug.Log("Hover");
37:                break;
38:            case InteractableState.Select:
39:                Debug.Log("Select");
40:                break;
41:            case InteractableState.Disabled:
42:                Debug.Log("Disabled");
43:                break;
44:        }
45:    }
46: }

```

このコードでは、IInteractableView インターフェースを実装したオブジェクトの状態変化を検知しています。ここで、IInteractableView を実装したクラスとは、Hand Grab Interactable などのことです。このインターフェースでは、Interactable の状態変化を

検知し、WhenStateChanged イベントを発火させる仕組みが用意されているため、これを購読する形で状態変化を検知できます。なお、前述した Interactable Unity Event Wrapper コンポーネントを使用すれば、コードを書かずに状態変化を検知することも可能です。

第 5 章

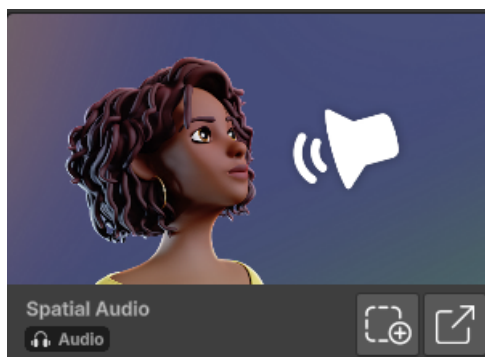
VR 空間と Audio / UI

この章では Audio や UIなどを、VR 空間に配置する方法について解説します。VR 空間に配置することで、ユーザーはより没入感のある体験を得ることができます。

5.1 VR 空間上で音を鳴らす

VR 空間上では、音を鳴らすために Spatial Audio（空間オーディオ）を使用する必要があります。Spatial Audio を使用することで、ユーザーの位置や向きに応じて音の聞こえ方が変化し、よりリアルな音響体験を提供できます。

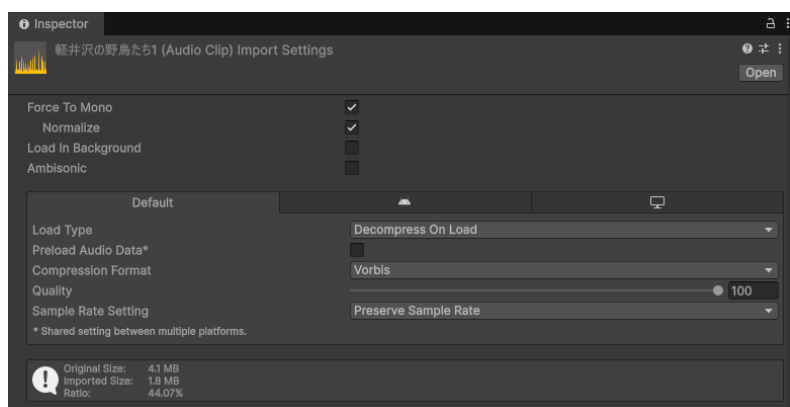
まずは Building Blocks から Spatial Audio をシーンに追加しましょう。



▲ 図 5.1 Spatial Audio の追加

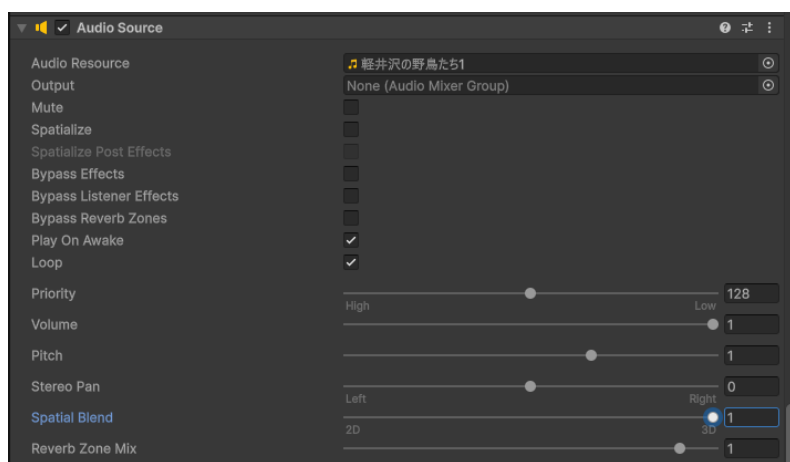
今回は BGM として、適当なフリー素材を使用します。なお、Spatial Audio として使用する音源は、Force To Mono を有効化して、モノラルに変換しておく必要があるので

注意してください。



▲図 5.2 音源の設定

Spatial Audio の Audio Source に、先ほどの音源を指定します。なお、Spatial Audio の Spatial Blend は、完全に 3D 空間で鳴らしたい場合は 1 に設定してください。一方で UI の効果音など、ユーザーの位置に関わらず聞こえるようにしたい場合は 0 に設定しましょう。



▲図 5.3 Audio Source の設定

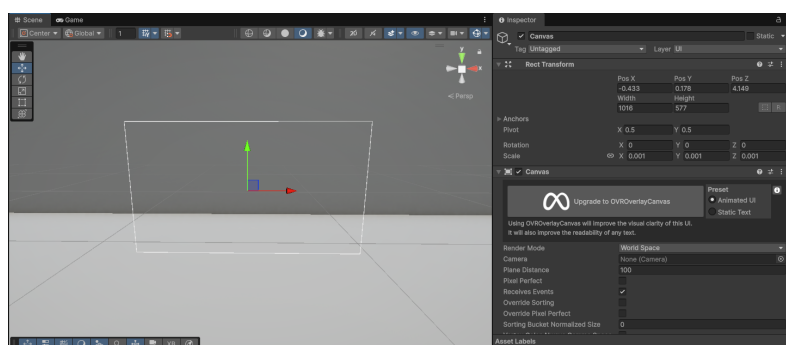
Audio Source の Play On Awake を有効化しておけば、シーン開始と同時に BGM が

流れ始めます。試しに実機で確認してみてください。

5.2 UI の設置

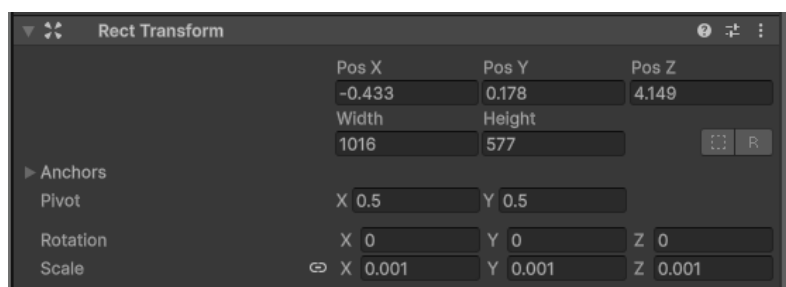
XR 空間では、特殊なコントローラーに対応したポインタブルなキャンバスを作成することで、ボタンクリックなどのインタラクションを実現します。インタラクションが必要ない場合、Canvas の Render Mode を World Space に変更するだけで VR 空間上に表示可能になります。

ここでは、インタラクト可能な UI を実装する方法を説明していきます。
まずはキャンバスを設置し、Render Mode を World Space に変更します。



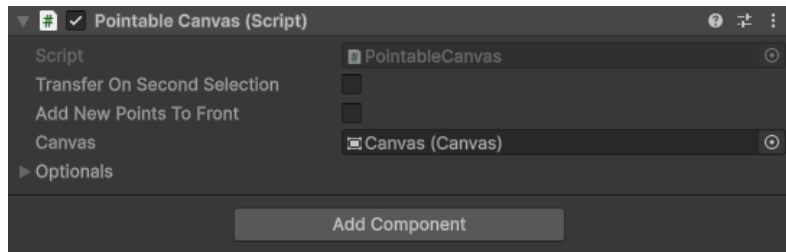
▲図 5.4 キャンバスの設置

そのままのサイズだとワールド空間に対して大きすぎるので、Scale を 0.001 倍程度に縮小しておきましょう。



▲図 5.5 キャンバスの縮小

続いて、キャンバスに Pointable Canvas コンポーネントを追加します。対象のキャンバスには、自身の Canvas コンポーネントを指定しておきましょう。



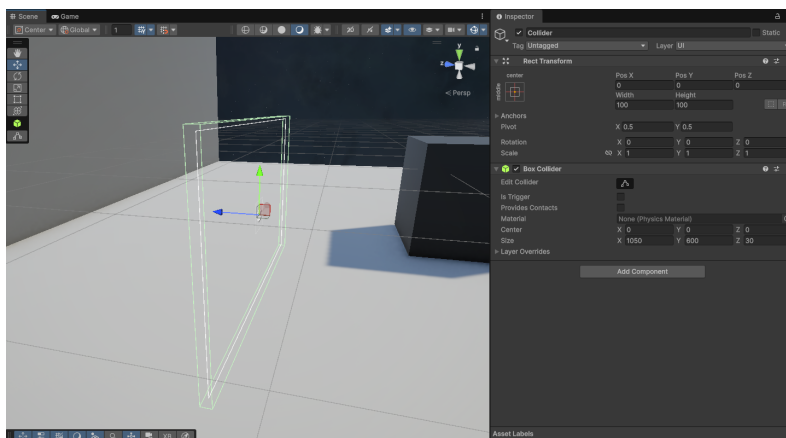
▲図 5.6 Pointable Canvas の追加

続いてキャンバスに当たり判定をつけていきます。キャンバスに空の子要素を追加し、Collider とします。



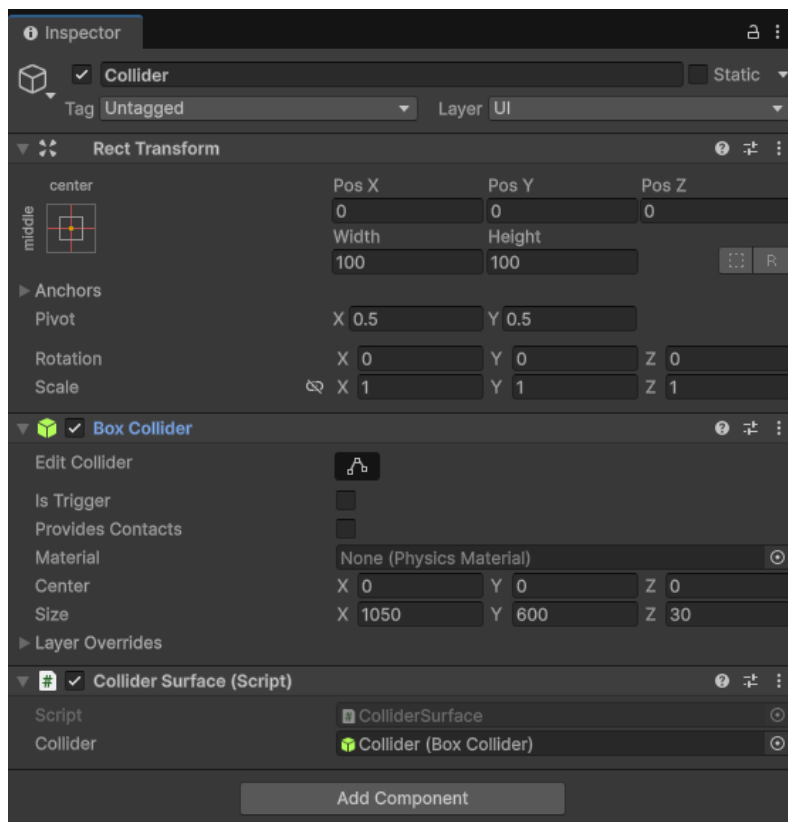
▲図 5.7 Collider の追加

この子要素には Box Collider を追加しますが、この時点だと、Canvas とのスケールリングの差でコライダーが見えないほど小さいため、サイズを大きくしておきましょう。大きさはだいたいキャンバスを覆う程度であれば問題ありません。



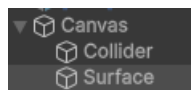
▲図 5.8 Collider のサイズ調整

Collider にはさらに Collider Surface コンポーネントを追加し、自身を Collider に割り当てます。



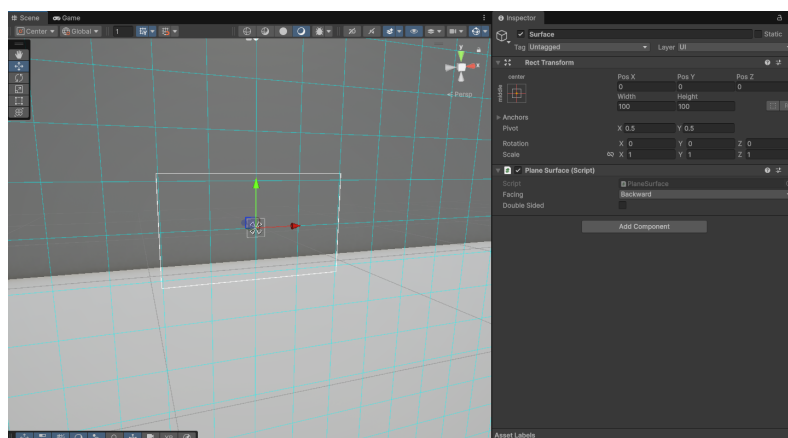
▲図 5.9 Collider Surface の追加

続いて同じく、キャンバスに空の子要素を追加し、Surface と名前をつけます。



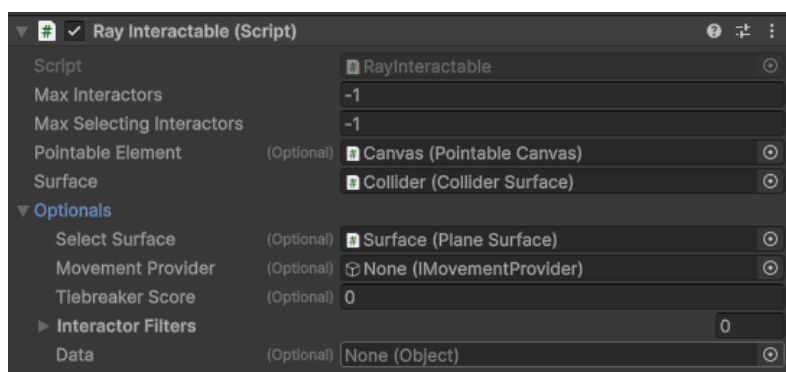
▲図 5.10 Surface の追加

このオブジェクトに対し、Plane Surface コンポーネントを追加します。



▲図 5.11 Plane Surface の追加

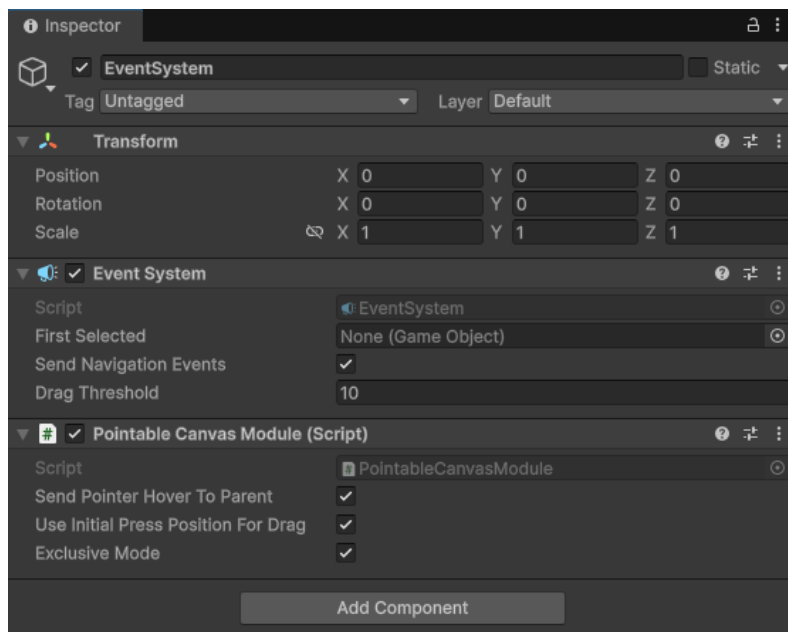
このコンポーネントが追加出来たら、Canvas に RayInteractable コンポーネントを追加しましょう。参照設定は以下です。* Pointable Element -> Canvas オブジェクト* Surface -> Collider オブジェクト* Select Surface -> Surface オブジェクト



▲図 5.12 RayInteractable の設定

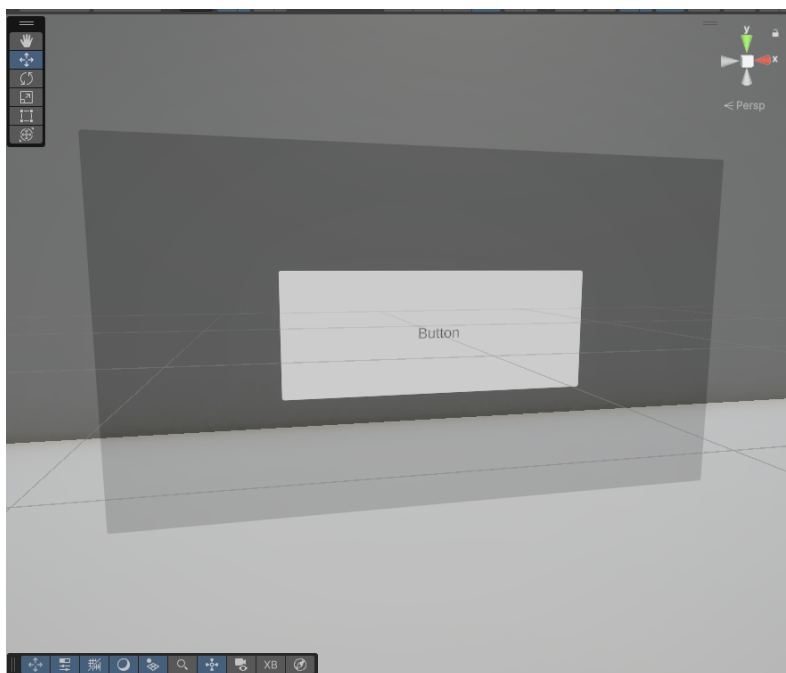
これにてキャンバスの設定は完了しました。プレハブ化しておけば、他の UI を作る時も同じ設定をする必要がなくなります。

次に Event System の設定に移ります。イベントシステムにデフォルトで含まれる Input System UI Input Module を削除し、代わりに Pointable Canvas Module を追加します。



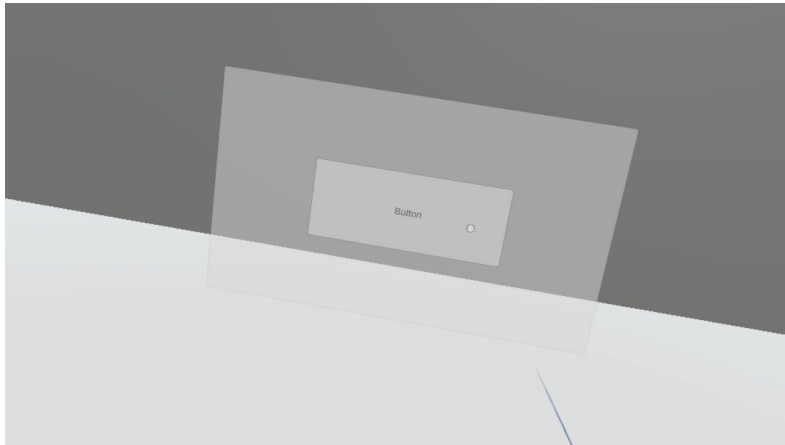
▲図 5.13 Event System の設定

あとは通常通り、キャンバスにパネルやボタンなどの要素を追加してみましょう。



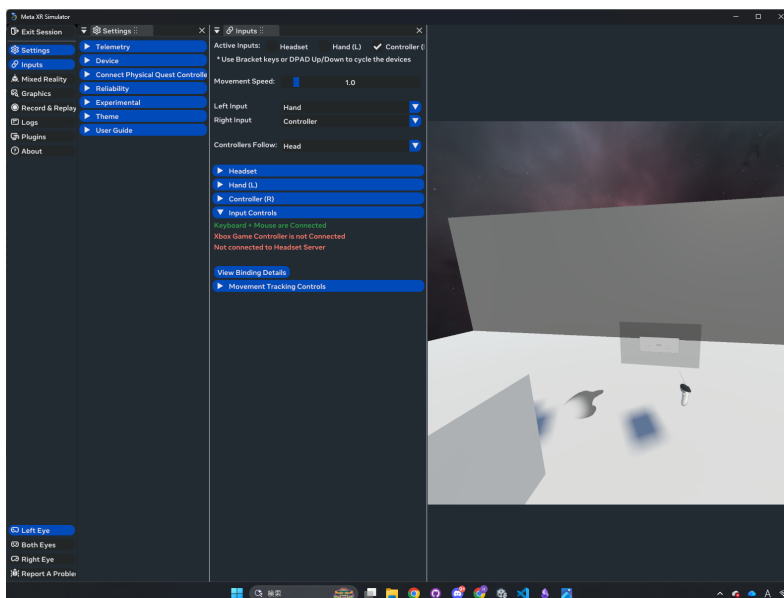
▲ 図 5.14 UI 要素の追加

この状態でビルドし、実機で確認すると、図 5.15 のように、コントローラーから Ray が伸びて、キャンバスの要素を選択できます。



▲図 5.15 UI の動作確認

XR Simulator でも動作確認可能である。



▲図 5.16 XR Simulator での動作確認

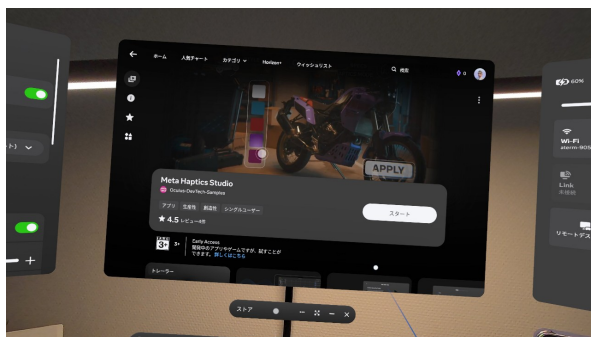
第 6 章

Haptics

VR において、触覚フィードバックは非常に重要な要素の一つです。この章では、Meta XR SDK における触覚フィードバック（Haptic）の基本的な使い方を説明していきます。

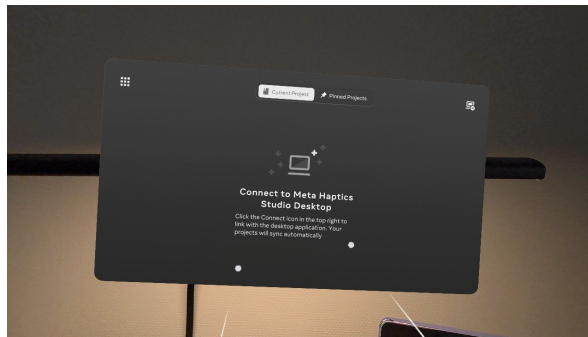
6.1 Haptic Studio

Haptic を準備するには、HapticsStudio が必要になります。
以下のサイトからダウンロードしてください。（使用バージョン：2.2.0）<https://developers.meta.com/horizon/downloads/package/meta-haptics-studio-win>
Meta3 にも Haptics Studio をインストールしましょう。



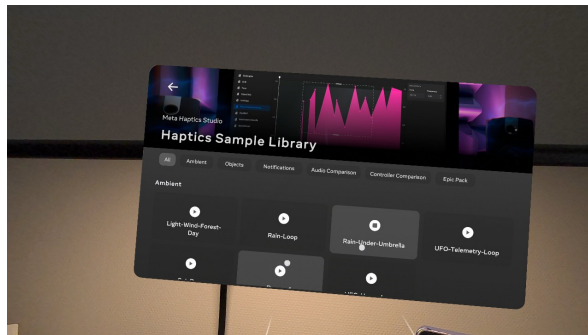
▲図 6.1 Meta3 での Haptics Studio のインストール

起動するとこのような画面が表示されます。



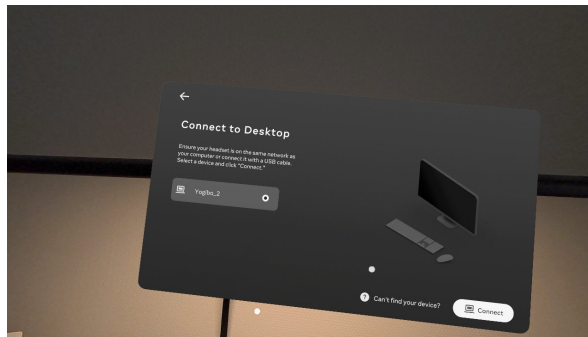
▲図 6.2 Haptics Studio の起動画面

このアプリ単体でもサンプルを再生して、Haptic を体験することができます。



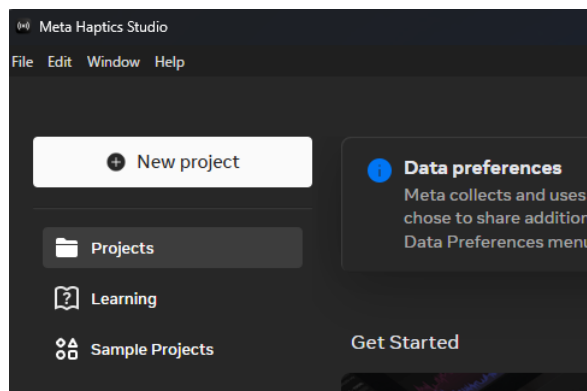
▲図 6.3 Haptics Studio のサンプル再生

続いて、PC と Meta3 を連携して、Haptic を作成する方法を説明します。PC と Meta3 を USB で接続し、Meta3 側で Connect を押してください。これで PC と Meta3 の接続が完了します。



▲図 6.4 PC と Meta3 の接続

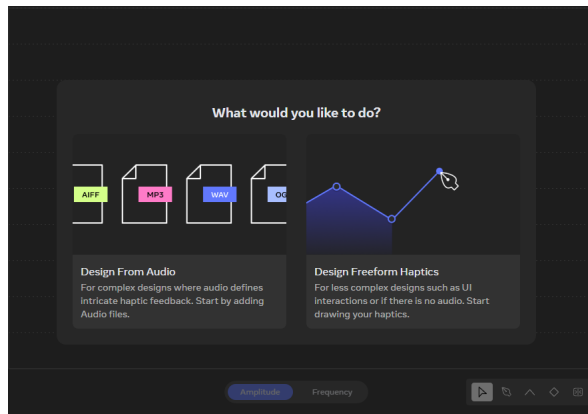
PC 側での操作に移ります。Haptics Studio の左上のメニューから、New Project を選択して新規プロジェクトを作成します。



▲図 6.5 新規プロジェクトの作成

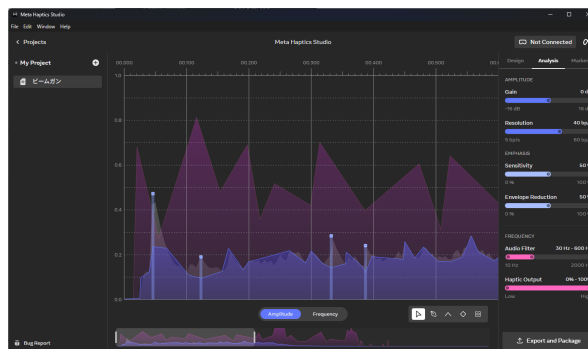
制作をはじめるには 2 種類の方法があります。

- * Design From Audio：好きな音源を取り込み、その音源から Haptic を生成する。*
- Design Freeform Haptics：波形を直接編集して Haptic を生成する。



▲図 6.6 Haptic の制作方法の選択

今回は、Design From Audio を選択して、ビームガンの発射音を収録した「ビームガン.mp3」を読み込みました。



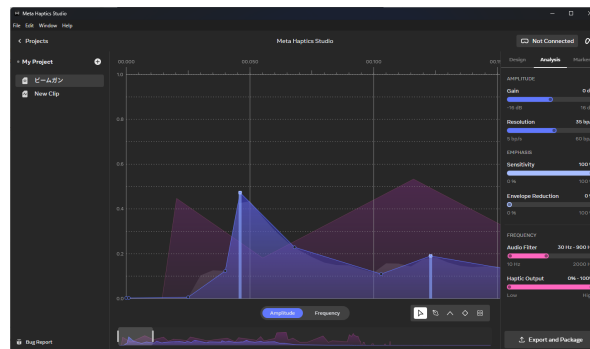
▲図 6.7 音源の読み込み

各要素について簡単に説明します。* Amplitude：振幅。Haptic の強さを表すパラメーター。* Frequency：周波数。Haptic の振動の速さを表すパラメーター。

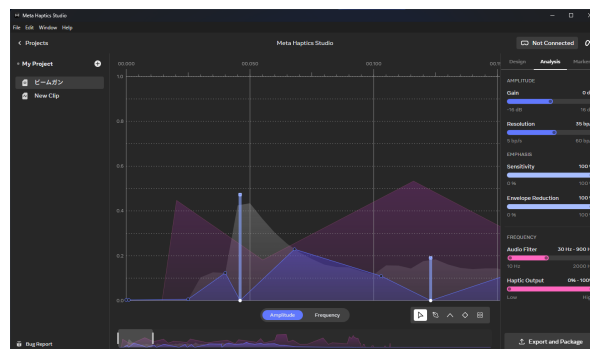
- Gain：入力音量。Haptic 全体の強さを調整するパラメーター。
- Resolution：解像度：Haptic の振動の細かさを調整するパラメーター。
- Emphasis：強調ポイント。Haptic の中で特に強調したい部分を設定できる。
- Sensitivity：感度。音声データの素早い変化から、強調ポイントをどれくらい抽出

するかを調整するパラメーター。

- **EnvelopeReduction**：強調ポイントの振幅を調整する。0% で強調ポイントと振幅が一致する。逆に 100% に近づくにつれ、強調ポイントの振幅は 0 に近づく。強調ポイント自身の振幅が小さくなるわけではないため、このパラメータを小さくすると強調ポイントが目立ちづらくなり、逆に大きくすると目立つようになる。

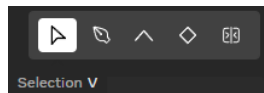


▲図 6.8 Envelope Reduction 0%



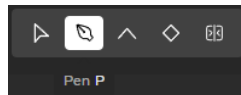
▲図 6.9 Envelope Reduction 100%

すでに設置されているポイントを編集する場合は Selection Tool を選択。



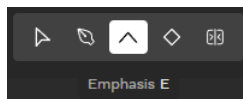
▲図 6.10 Selection Tool の選択

新しいポイントを追加する場合は Pen Tool を選択。



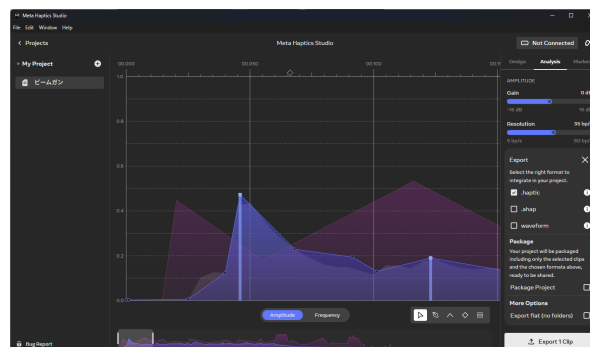
▲図 6.11 Pen Tool の選択

強調ポイントを手動で追加する場合は Emphasis Tool を選択。



▲図 6.12 Emphasis Tool の選択

完成した Haptic は、Export から .haptic ファイルとして出力できます。それ以外の形式は Unity では利用できません。



▲図 6.13 Haptic の出力

6.2 Haptic の再生

前節で作成した .haptic ファイルを Unity にインポートして、実際に再生してみましょう。インポートした .haptic ファイルは、Scriptable Object として扱われます。



▲ 図 6.14 インポートした .haptic ファイル

この Haptic を再生するには、Haptic Clip Player をインスタンス化し、Haptic Clip として渡せばよいです。

▼リスト 6.1 Haptic Clip Player のコード例

```

1: using UnityEngine;
2: using Oculus.Haptics;
3:
4: /// <summary>
5: /// HapticClipを再生するサンプル。
6: /// </summary>
7: public class HapticPlaySample : MonoBehaviour
8: {
9:     [SerializeField]
10:     private HapticClip _hapticClip;
11:
12:     private HapticClipPlayer _hapticClipPlayer;
13:
14:     private void Awake()
15:     {
16:         _hapticClipPlayer = new HapticClipPlayer();
17:     }
18:
19:     public void PlayHaptic(Controller controller)
20:     {
21:         _hapticClipPlayer.clip = _hapticClip;
22:         _hapticClipPlayer.Play(controller);
23:     }
24: }

```

ほとんど AudioClip を再生するのと同じような感覚で Haptic Clip を再生できることがわかりますね。HapticClipPlayer の Play メソッドの引数には、どちらのコントローラーで Haptic を再生するかを指定します。

▼リスト 6.2 Controller の定義

```
1: public enum Controller
2: {
3:     Left,
4:     Right,
5:     Both
6: }
```

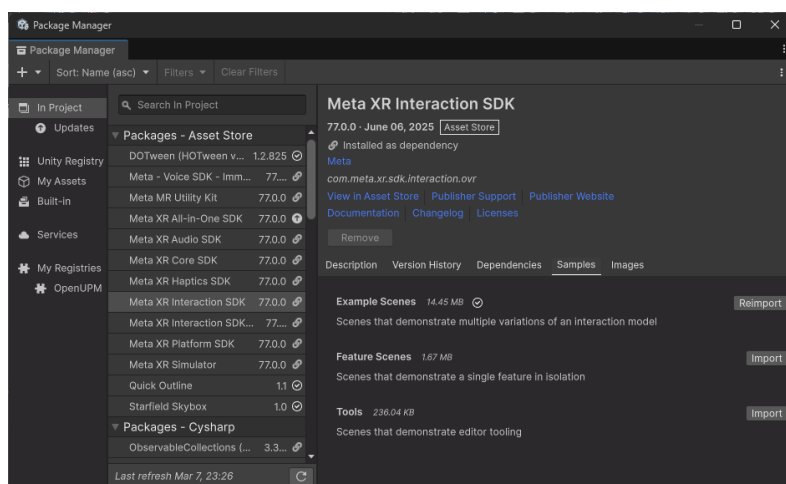

第7章

SDK のサンプルシーン

最後におまけとして、Meta XR SDK に同梱されているサンプルシーンを紹介します。非常に参考になる内容が多いので、ぜひ一度は目を通してみてください。

7.1 インポート

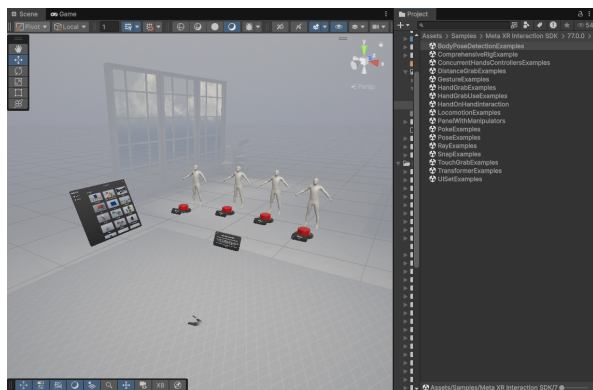
Package Manager から、Meta XR Interaction SDK の Samples を開き、Example Scenes をインポートしてください。



▲図 7.1 Example Scenes のインポート

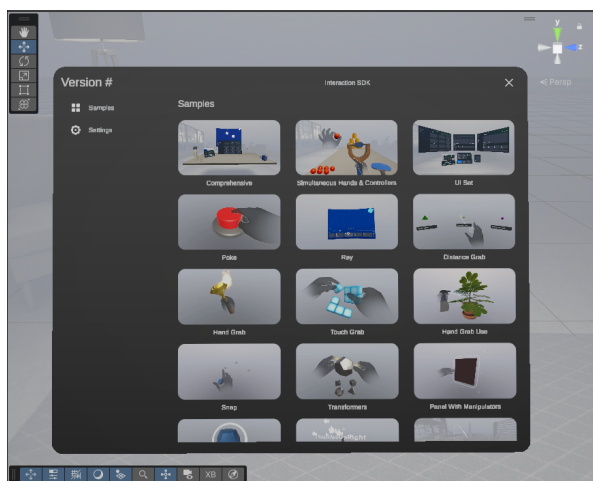
7.2 体験

Exsample Scenes には、様々なインタラクティブ可能なシーンが同梱されています。



▲図 7.2 とあるシーン

各シーンをビルド対象に追加しておくことで、シーン内に設置された UI を通じて、SDK の様々な機能を体験することができます。

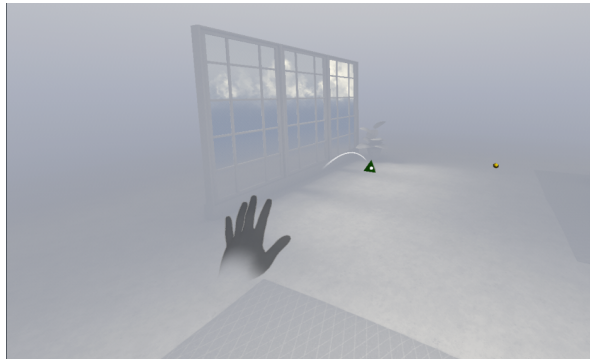


▲図 7.3 シーン遷移用 UI

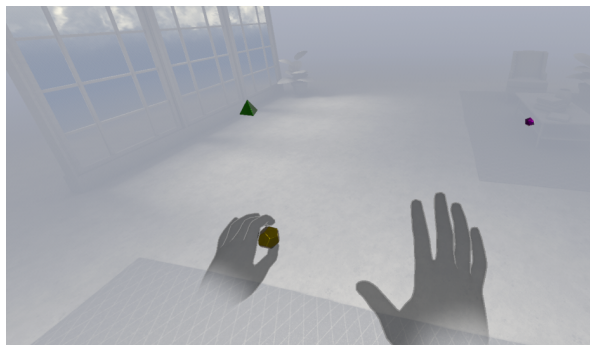
7.3 Distance Grab Scene

例として、本資料で紹介した Grabbable の別種を体験できるシーンを紹介します。

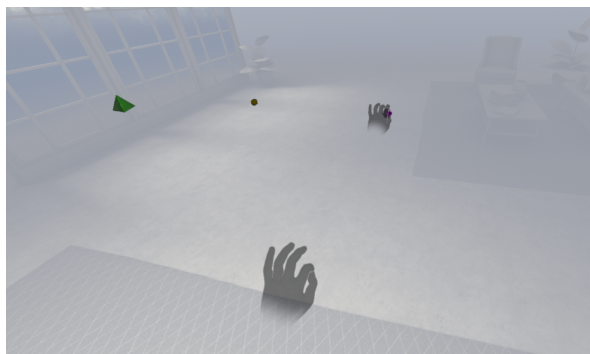
Distance Hand Grab Interactable というコンポーネントを使えば、遠く離れているオブジェクトに対しても、掴む動作を行うことができます。



▲図 7.4 遠くにあるものをその場から動かせる



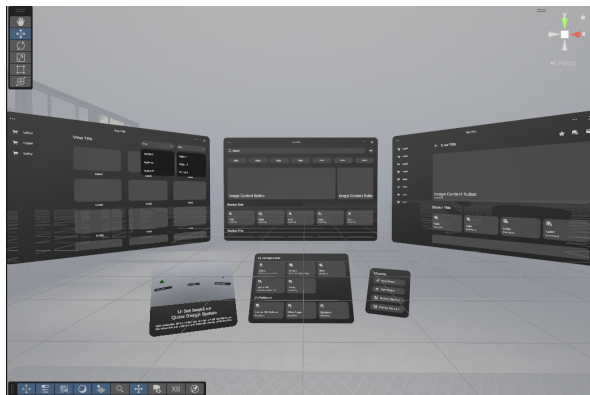
▲図 7.5 遠くにあるものを近くに引き寄せられる



▲図 7.6 遠くにあるものを複製された手で操れる

7.4 UI Set Examples Scene

Meta 公式の UI Set を使用したシーンも同梱されています。UI Set を使用することで、VR 内での UI 操作を簡単に実装することができます。



▲図 7.7 UI Set を使用したシーン

このほかにも様々な機能が存在するので、ぜひ自分の目で確かめてみてください。

CGP VR 開発 入門書

2026 年 3 月 10 日 CGP VR 開発 入門書 v1.0.0

著 者 CGP プログラマー部門

編 集 Yoshioka

発行所 CGP

(C) 2014-2026 CGP